

Byzantine Broadcast with Unknown Participants

Wonseok Choi* Ran Cohen† Juan Garay‡ Nikos Skoumios§ Vassilis Zikas¶

Abstract

A sender wishes to consistently broadcast a message on the dark web, so that whoever is around and active will agree on it even when the sender is malicious. No assumptions on the number of honest parties, or blockchain-style “tricks”—like balanced resource-allocation (e.g., hashing power or stake ownership)—can be made.

The above is an instance of *Byzantine broadcast (BB)* in the *unknown-participants* setting (“UP Broadcast” for short). Despite four decades of extensive research on dishonest-majority BB, all existing approaches (e.g., the well-known Dolev-Strong protocol) fail to solve this problem, as they crucially rely on *knowing* the number of protocol participants—or the make blockchain-style assumptions on available resources. The challenge, which might appear as an inherent limitation, is that without any such assumption malicious parties can join the protocol at any point during its execution, making it arduous for other parties to terminate without violating consistency. So one might wonder: *Is this even possible?*

In this work, we provide the first definitions of UP Broadcast that incorporate both static and dynamic participation and corruption of arbitrary many parties. Interestingly, even formally defining the problem turns out to be non-trivial as one needs to deviate from the model used in classical BB approaches. We then provide the strongest possible (and in our opinion, unexpected) answer to the above question: *Yes, it is!* We provide a polynomial-time *deterministic* UP Broadcast protocol. In the process we also solve *UP Interactive Consistency*, which corresponds to the multi-sender version of the problem. Our constructions are in the standard, synchronous model of protocol execution, and they offer consistency and validity guarantees to every party who is present throughout the protocol execution.

We next turn to the question of round complexity and prove that our protocols are optimal against adversaries who can corrupt arbitrarily many parties; this optimality applies even to *randomized* protocols. Finally, we ask, what if parties join in the middle of the protocol execution? We provide a negative result for unrestricted *dynamic participation*; on the positive side, we devise definitions that offer best-possible guarantees (also to such “late” parties), and present corresponding constructions that remain round-optimal.

*DGIST. E-mail: wonseok@dgist.ac.kr.

†Reichman University. E-mail: cohenran@runi.ac.il.

‡Department of Computer Science and Engineering, Texas A&M University. E-mail: garay@tamu.edu.

§University of Washington. E-mail: nskoum@cs.washington.edu.

¶School of Cybersecurity and Privacy, Georgia Institute of Technology. E-mail: vzikas@gatech.edu.

Contents

1	Introduction	1
1.1	Technical Overview	2
1.2	Related Work	6
2	Modeling Broadcast with Unknown Participants	9
2.1	The Unknown-Participants Model	9
2.2	Byzantine Broadcast in the UP Model	12
3	Active-Parties Set Agreement Protocol	14
3.1	Adapting Dolev-Strong to the UP Model	15
3.2	The APSA Protocol	15
3.3	APSA Protocol Analysis	16
4	Interactive Consistency with Unknown Participants	19
4.1	Building on the APSA Protocol	20
4.2	The UPIC Protocol	20
4.3	Interactive Consistency Protocol Analysis	21
5	Byzantine Broadcast with Unknown Participants	23
6	Round-Complexity Lower Bounds	24
6.1	A Lower Bound for UP Broadcast	24
6.2	Lower Bounds for APSA and UPIC	27
7	Late-Joining Parties	28
7.1	Security for Late-Joining Receivers	28
7.2	Security with a Late-Joining Sender	32
	References	36

1 Introduction

Byzantine agreement (BA) [PSL80, LSP82] is one of the most fundamental primitives in fault-tolerant distributed computing. It allows a set of parties to reach agreement on their inputs in an adversarial setting where a malicious attacker that controls some of the parties is trying to prevent them from reaching their goal. BA comes in two flavors, *Consensus* and *Broadcast*.¹ In Consensus, every party has an input and the goal is to agree on the output (*consistency*), such that if all honest parties start the protocol with the same value, then this value is agreed upon at the end of the protocol (*validity*). In Broadcast, on the other hand, only a single distinguished party, called the *sender*, has an input, and the requirements are that all honest parties agree on the same value, which equals the sender’s input in case the sender is honest throughout the protocol.

The bulk of the literature on BA considers a (typically known) set of n parties who are connected with each other via a complete network of reliable and authenticated channels. In this setting, BA (both Consensus and Broadcast) can be achieved if and only if at most $t < n/3$ of the parties are malicious, unless a trusted setup (e.g., a public key infrastructure, PKI) for digital signatures is assumed [LSP82, FLM86, Bor96]. Assuming such a PKI setup, one can achieve Broadcast with arbitrary many malicious parties $t < n$ [DS83], and Consensus for up to $t < n/2$ corruptions, a bound also known to be tight [Fit02].

The assumption of a known universe of participants has been implicit in the BA literature—and more generally in the (fault-tolerant) distributed computing literature—for over four decades. The advent of blockchains, however, fueled by the massive success of Bitcoin, opened up several new horizons and pointed to their usefulness for the novel realization of distributed systems. In particular, a common assumption in what is frequently referred to as the “permissionless” model, is that the exact upper bound on the number of participants in the protocol is not known, and, further, that parties might join and leave during the execution of the protocol (a property often referred to as “dynamic availability/participation” [PS17, GKL17, BGK⁺18]). These assumptions have become mainstream in the blockchain setting and novel instantiations of state-machine replication [Sch90] (a.k.a. “Nakamoto Consensus” or “Ledger Consensus”).

This leads to the natural question of understanding the feasibility of Byzantine agreement in a model where the set or number of participants is **unknown**; in the sequel we will refer to this setting as the *unknown participants* (UP) model. The common denominator in current blockchain systems is to assume that the majority of some resource is in honest hands—e.g., hashing power in Proof-of-Work-based system or amount of stake in a Proof-of-Stake-based system. This so-called *resource-restricted setting* [GKO⁺20], allows for replacing the honest majority of parties assumption by an honest majority of resources assumption in the UP model, and making such assumptions on the parties’ resources has proven sufficient for dealing with unknown (and/or fluctuating/dynamic) participation [PS17, GKL17, BGK⁺18]. However, the question of whether BA (in particular Broadcast) is possible in this UP model without assuming an honest majority of neither resources nor parties has not to our knowledge yet been addressed. This is the question we tackle in this work:

Can a sender (consistently) broadcast a value to whomever is available, without a priori knowledge of the identities of the available receivers, and without any assumptions on the number of corruptions or balance of resources?

Our results in a nutshell. The challenge posed by the above question should be apparent to anyone who has worked in this space: How can I even decide that I have received messages from

¹The BA literature often uses the term BA in a somewhat ambiguous manner. Here we will use the terms consensus and broadcast to make explicit which of the two flavors we are referring to.

sufficiently many parties to ensure I make a consistent decision? This basic question is “engraved” into the problem and might lead one to believe that there is very little one can do here. But, surprisingly, we prove in this paper that this is not the case, by presenting feasibility results that achieve in this UP model analogous guarantees as what can be achieved in the known-participants model. Further, our protocols are adaptively secure and, as we show, achieve optimal round complexity.

In more detail, in this work we put forth a cryptographic model for Byzantine Agreement with *unknown participants (UP)*. Our model addresses several definitional challenges that are introduced when such a model is combined with computational assumptions—e.g., allowing a potential participant pool size that is exponential in the security parameter. We also provide security definitions for Broadcast in this model, which we term *UP Broadcast*. These definitions are far from trivial—even in the static adversary case—due to the need to capture different guarantees depending on when parties join and/or when they might get corrupted, which are execution-time-defined (in contrast to the standard BA definitions where the number of parties is defined at the beginning and known to the protocol).

We then turn to the question of whether UP Broadcast is feasible with a dishonest majority. Somewhat surprisingly, we answer this question in the affirmative, and even provide protocols with tight round complexity. In fact, our protocols are deterministic with simultaneous termination, while the lower bounds hold even for randomized broadcast protocols.

1.1 Technical Overview

Our work paints a detailed picture of the feasibility landscape of UP Broadcast in the dishonest-majority setting with arbitrarily many corruptions, which includes several protocols with new technical ideas and corresponding lower bounds. We begin by presenting a snapshot of these results and techniques, and outline the challenges that the UP model presents.

The unknown-participants model. One of the technical challenges in this work is formalizing the UP model itself; contrary to the other models that assume an honest majority of the parties or resources, we aim to remove such assumptions. Without such assumptions, the set of all potential protocol participants which we call the universe $\mathcal{U}$, can potentially be exponentially larger than the actual number of protocol participants. (We note that this deviates from the so-called *ad-hoc* model for secure multi-party computation [BGIK16, BIK17], which assumes a polynomial universe of parties.) Furthermore, the parties in the universe have no *a priori* knowledge about each other. We carefully formalize our UP model to appropriately define Byzantine broadcast in our setting.

Denote by κ the security parameter. The universe $\mathcal{U}$ of parties can be exponentially large in κ ; each party in $\mathcal{U}$ has a unique ID from $\{0, 1\}^{\text{poly}(\kappa)}$. Inspired by simulation-based security definitions, we consider a PPT environment $\mathcal{Z}$ and a PPT adversary $\mathcal{A}$. The environment $\mathcal{Z}$ can *activate* polynomially many parties in $\mathcal{U}$ that will actually participate in the protocol, and the adversary $\mathcal{A}$ can corrupt activated parties and control them in an arbitrary, malicious manner. One caveat is that set of active parties (and, in particular, the set of honest parties) can change even within an execution of the protocol. Therefore, as opposed to standard definitions of Broadcast, we *cannot* consider a fixed and known set of parties in the protocol; rather, we refer to specific protocol *executions*. We denote an execution of the protocol by $\mathcal{E}$, and emphasize again that the number of parties and their identities can change between one execution and the next.

Another caveat is that due to the exponential size of the universe, standard models of a public key infrastructure (PKI) cannot be used. Indeed, no party (not even the environment and the adversary) can store the public keys of all parties; stated differently, any party that can store such

a PKI can break the security of the underlying signatures. However, although the potential pool of parties is exponential, it is guaranteed that only polynomially many parties will be activated during the protocol (since the environment runs in polynomial time). Therefore, we define an *ad hoc PKI* model, where a trusted *certificate authority* (CA) certifies the public keys generated by the activated parties. See further details in Section 2.1.

Modeling UP broadcast. Recall that in the UP setting the set of participating parties may change in each execution (as opposed to standard BA); this must be taken into account when defining UP Broadcast. This is not too challenging in the static setting (which is used for our lower bounds), where the environment activates all parties and the adversary chooses whom to corrupt before the protocol begins. We require that all active parties that are honest in the execution agree on their output, which is the sender’s input if the sender is *activated* and honest.

Things are much more subtle in the dynamic setting (used for our protocol constructions). Roughly speaking, we require that parties which are activated from the beginning and never get corrupted by $\mathcal{A}$, shall reach agreement within a polynomial number of rounds. In addition, if the sender is activated from the beginning and does not get corrupted, then this common output must equal to the sender’s input. In Section 2.2, we discuss subtle challenges with this model.

Problems with existing approaches. One can easily observe that classical dishonest-majority broadcast protocols (e.g., Dolev-Strong [DS83], *DS-Broadcast* for short) cannot satisfy the UP Broadcast definition. For example, it is crucial in DS-Broadcast that all honest parties run for $t+1$ rounds (for $t < n$) if security is against any malicious subset, but how can this be done if n (and t) are unknown?

One might be tempted to assume that the above shortcoming of Dolev-Strong in the UP model can be addressed by so-called early stopping protocols [DRS90]. In a nutshell, such protocols aim to reduce the number of rounds from a function of just the upper bound, t , on the number of corrupted parties, to a function of the *actual* number of faults—i.e., the number of parties that actually misbehave. Examples of such results exist both in the information-theoretic setting [BGP92, AD15a], which show protocols terminating in $\min(t+1, f+2)$ rounds assuming $t < n/3$, and in the cryptographic setting [Per85, LN24]— $\min(t+1, f+2)$ rounds assuming $t < n/2$, and $O(\min(t+1, f^2))$ rounds assuming $t = n-1$, respectively.

However, to our knowledge, all early-stopping BA literature suffers with similar issues as the Dolev-Strong protocol when run in the UP model. In particular, classical early-stopping protocols aim to correlate their termination with a runtime-identifiable property, namely, whether or not consensus has been achieved. Now, detecting consensus requires an honest majority which makes their approach inapplicable here. In fact, to our knowledge, with the exception of [LN24, DKL25], all such protocols assume an honest (super-)majority—in addition to the total number of parties being common knowledge. Notwithstanding, the methodology of [LN24, DKL25] can also not be used here as it relies on the well-known phase-king approach, which assumes a natural (agreed-upon) ordering of the players and picks a number of kings according to this ordering; again without knowledge/agreement on the active players, it is unclear if or how such a king-selection can be realized—in fact, even randomizing the selection does not seem to help as we do not have (assumptions allowing) cryptographic sortition.

The above discussion highlights that the problem with classical approaches lies at a much deeper level. Specifically, already the size of $\mathcal{U}$ (which recall is exponential in κ) poses a challenge: Existing round-complexity lower bounds for *deterministic* broadcast show linear dependency on the number of corruptions [DS83]. The same holds for *randomized* broadcast with an arbitrary

number of corruptions [GKKO07, CPS20].² Hence, in the dishonest-majority case, this translates to linearly many corruptions in the number of active parties that participate in the protocol. Since the parties (i.e., the protocol) have no information on this number (which is not even fixed when the protocol begins—only when it ends), to ensure termination the number of rounds would need to be linear in $|\mathcal{U}|$, which is super-polynomial in the security parameter κ . But this means that no such existing broadcast protocol can retain its security, since they all must rely on a trusted setup and computational assumptions, which can be broken by a super-polynomial adversary.³

Active-parties set agreement. The above limitation of the classical (known and polynomially many participants) model sheds light on what needs to be done in the UP model:

Enable parties to agree on (the cardinality of) the set of active parties.

But how can we do that in the UP model, where an adversarial (worst-case) environment might activate new parties at will during the protocol execution? We achieve this goal by a new primitive we introduce, which we call *active-parties set agreement* (APSA) (see Section 3). In a nutshell, this primitive allows parties to agree on a set S that includes every party that has been active (and honest) since the beginning of the protocol’s execution and until its completion, potentially along with some additional corrupted parties who have joined the execution in the middle.

To understand our key technical insight in solving APSA, it is useful to recall the structure and key idea behind DS-Broadcast: The protocol starts with the sender sending its signed input to every party; then in every following round r , each party checks that it has received a strong enough “certificate” (effectively, a chain including signatures from $r - 1$ different parties) on the sender’s input, who adds its own signature and forwards it. The key observations are that (1) this forwarded certificate will be perceived as valid by any honest party in the next round, and (2) since there are at most n parties, the last convincing round will be n (because anyone convinced in that round will see a certificate signed by all other parties, and hence also by other honest parties).

A strawman approach to APSA would be to have every party act as the sender and have the parties agree on the set of senders. This is problematic in many ways: First, we want to agree on parties that have been honest from the beginning—in fact, as we argue in Section 7, we cannot offer validity guarantees for senders who are not honest from the beginning of the execution. One might be tempted to think that requiring the parties to sign also the current round solves this problem, but this is not the case since, for corrupted senders, the adversary can create fake certificates for corrupted parties. We resolve this issue by a more sophisticated certificate generation and verification procedures that take into account not only the set of signatures included in a DS-Broadcast-like certificate, but also their *order*. We note in passing that a challenge here is to make this mechanism efficient, i.e., avoid storing and forwarding all received permutations which would yield an LSP-like “exponential information gathering” approach [LSP82, BDDS92].

The second problem is even more challenging. Since we do not know the total number of parties, how can the protocol terminate in a way that guarantees that no other party continues (and potentially adds more parties to its local set of active parties, which would violate consistency)? We resolve this by a new mechanism that detects when the environment has stopped activating new parties for a sufficiently long interval, thus ensuring that parties cannot get confused.

²When a constant fraction of the parties is guaranteed to be honest, the lower bound for randomized protocols is poly-logarithmic [GKKO07, CPS20]; however, in the UP model this is a rather strong assumption that we do not wish to uphold.

³In fact, this super-polynomial gap between the number of potential parties and active parties is not only the natural choice for the UP model, but is also a key point that differentiates the UP setting from the classical (known-participants) one, and eliminates the trivial solution of running DS-Broadcast for $|\mathcal{U}|$ rounds.

UP Interactive Consistency. The above APSA protocol is at the heart of our UP Broadcast construction. Building on that protocol, we construct an Interactive Consistency (IC) protocol [PSL80] in the UP model (UPIC). IC is a well-known primitive which essentially corresponds to parallel broadcast (without input independence): Each player has an input to broadcast and the output is the ordered concatenation of the broadcast values (where corrupted parties might bias their broadcast values based on the honest players' inputs). One solution here would be to run APSA followed by DS-Broadcast for each sender, where parties not in the APSAs output are ignored. This would indeed solve the problem, since our guarantees are for parties who have been active since the beginning of the protocol execution, but it would double the number of rounds. Instead, in Section 4 we design a UPIC protocol with the same (and as we prove, optimal) round complexity, by a (white-box) revision of APSA which allows the parties to agree not only on the set S of active parties, but also on an input for each party in S , where validity is guaranteed for the inputs of honest parties in S .

Constructing UP Broadcast. Having UPIC we can very easily obtain a UP Broadcast protocol, since IC is strictly more general than Broadcast: Have every party run IC, where the sender (if it is activated at the beginning) uses its input while everyone else just uses a default value. In fact, this reduction can also take care of an extra annoyance in the UP Broadcast definition, dealing with an *inactive sender* (i.e., a sender who is not active from the start, see Section 2.2): Such a sender, if honest, will ensure not to be included in the set S by remaining silent, and the protocol will ensure to output $\perp$ for a sender not in S ; see Section 5.

We note that all our protocols achieve their respective definition against adaptive and rushing adversaries. These definitions are the UP analog of the standard property-based definitions and do not guarantee *corruption fairness* [CGZ23]. Incorporating corruption-fairness and making the treatment simulation-based secure is an interesting research direction.

Round-complexity lower bounds. Any execution of our protocols above runs in a number of rounds, which is upper-bounded by the number of activated parties in the execution. We next ask: Can we do better?

To answer this question, we prove in Section 6 a strong impossibility result: If the adversary can corrupt any number of the active parties, then it can make the protocol run for the above number of rounds. We prove this result by a refinement of the lower bound by Garay *et al.* [GKKO07]: We create a chain of appropriately many scenarios, say k , where for $i \in [k]$, we can argue in the i^{th} scenario that some parties cannot output before i rounds (without violating security). Interestingly, requiring that inactive senders broadcast $\perp$ yields one more scenario than [GKKO07] (one where the sender is inactive) which increases the lower bound by 1. We note that similarly to [GKKO07], this proof also excludes randomized, probabilistic-termination protocols, thereby establishing the tightness of our UP Broadcast protocol in terms of round complexity.

Late-joining parties. The definition of UP Broadcast discussed so far gives no guarantees to late-joining parties, i.e., parties that are activated after the protocol starts and before it terminates. In Section 7, we study UP Broadcast that accommodates such late-joining parties. We consider two variants of the problem: *security for late-joining receivers* and *security for a late-joining sender*.

Informally, security for late-joining receivers (Definition 7.1) requires that they output consistently with parties that remain honest up to the termination point (of a party who has been honest and active from the beginning). In Lemma 7.2, we prove that no deterministic protocol can satisfy this definition when late-joiners can get activated at any time.

Nonetheless, we show how to modify our UP Broadcast protocol to achieve the next best thing: security with *selective abort* for late-joining receivers (Definition 7.3). Intuitively, this ensures that late-joiners either enjoy the same output guarantees with the parties that were honest from the beginning, or abort by recognizing that there was at least one corruption that prevents them from converging to the output. The security of our modified UP Broadcast with this guarantee is stated and proven in Theorem 7.4.

We next turn to security for a late-joining sender (Definition 7.5). Informally, we require that the input of a late-joining sender is included in the output of the parties. In Lemma 7.6 we show that this notion is too good to be true, and no deterministic protocol can satisfy it. However, in case the sender joins sufficiently early, before honest parties are about to terminate, our impossibility will not apply. A trivial protocol for the case that there is an upper bound R on the number of rounds that the sender may be late is to have the parties start the UP Broadcast protocol in round R . In Lemma 7.8 we show that this trivial protocol is the best one can hope for in terms of round complexity.

1.2 Related Work

The seminal results by Lamport, Shostak, and Pease [PSL80, LSP82] triggered a massive literature across multiple research communities studying various aspects of synchronous BA in several models. Here, we review the most relevant ones, skipping the classical feasibility results discussed in the previous section.

An important separating line in the BA literature, which is often blurred when feasibility results are discussed, is the one between classical consensus and *continuous consensus* (a.k.a. state machine replication (SMR) [Sch90], these days also called “ledger consensus”). Classical consensus is, as discussed above, a “single-shot” protocol, where parties start the protocol with some input and the goal is to terminate with a consistent output. Ledger consensus, on the other hand, is a continuous protocol, whereby parties have (potentially new) inputs as the protocol evolves. The goal is to continuously build a distributed ledger that every party agrees on, which ensures honest inputs are eventually accounted for. In fact, one might argue that the name “continuous consensus” is somewhat misleading as the underlying protocol is closer to Byzantine broadcast than consensus. Indeed, in the classical synchronous (fixed, known-participants) model, given a Byzantine broadcast protocol with simultaneous termination guarantee—i.e., where every party terminates in the same round—it is straightforward to construct a “continuous consensus” protocol: Simply have senders broadcast their input and order them according to the round when outputs are generated (where messages output in the same round are ordered according to an arbitrary rule, say, lexicographically). Importantly, such a reduction of consensus to broadcast is impossible. We refer to [GK20] for a survey and systematization of past and recent works on these two flavors of consensus.

BA with an honest majority. Alchieri et al. [ABdSFG08] considered a setting where an unknown subset from a fixed and known set of parties reaches consensus. Their setting is quite different than ours as they focused on perfect security over an asynchronous network, and could tolerate $t < n/3$ corruptions and connectivity of $2t+1$, where n is the actual number of participants. Khanchandani and Wattenhofer [KW21], considered classical consensus with unknown *number* of participants and constructed protocols where the participants have unique but not necessarily consecutive identifiers, and the number n of parties along with the number t of corruptions were both unknown (but fixed), but under the assumption that $t < n/3$.

Despite the similarity in the goals, the assumptions and treatment of [ABdSFG08, KW21] make it a far less surprising result. First, for the modeling aspects, they assume (information-theoretic) perfect security with $t < n/3$ parties and a purely static participation and adversary. This makes it a far easier goal: The underlying protocol first has everyone who is active contact everyone else in the network; since all $(2t + 1)$ honest parties will perform this operation, this step yields a good approximation of how many honest parties are around and how many of them are honest. This approximation can be improved by everyone relaying their views. The $t < n/3$ assumption is crucial here and makes the solution simple and elegant. However, none of these ideas can be used in the dishonest-majority setting, where any such approximation of corrupted parties will be inherently unreliable—since every adversarial party can communicate different sets of receivers. Indeed, one of the core technical challenges of the dishonest majority, is to obtain a consistent view of which parties are part of the broadcast. The above is the case even for static adversaries and environments, but the situation becomes even more challenging for the dynamic/adaptive scenarios discussed in this work. In addition to our setting being far more technically involved, the definitional aspects are also particularly more challenging. There are several reasons for this: First, without an honest majority, broadcast requires a (signatures) setup. Modeling such a setup for an unknown universe introduces additional challenges, creating the need for a novel, cryptography-inspired model of executions and corruptions.

The fully permissionless model of Lewis-Pye and Roughgarden. Lewis-Pye and Roughgarden [LR23a, LR23b] studied consensus in a setting, similar in spirit to ours, where at any point the protocol has no knowledge about which parties are currently running it. They proved several impossibility results, most relevant to our setting: (1) consensus cannot be solved facing any constant fraction of corruptions, (2) every deterministic consensus protocol must have an infinite execution facing any constant fraction of corruptions, and (3) in the dynamic-availability setting, there is no consensus protocol even when there are no corrupted parties at all. Yet, there are several core differences in both technical model and definitions. Our results neither contradict with nor are implied in any way by [LR23a, LR23b].

- First, these works consider the consensus problem that requires an honest majority even with a known set of parties [Fit02], whereas we consider the broadcast problem (and the multi-sender variant of interactive consistency); hence, their impossibility (1) does not apply to our case.
- Second, their impossibility (2) relies on two modeling choices: the first is to have a potentially *infinite* number of participants and the second on assuming idealized cryptography and requiring *perfect* correctness; in this case, they proved an FLP-style result of showing that there is a measure-zero set of executions that do not terminate. In contrast, we consider a more cryptography-compatible model with a *finite* (yet exponential-size) universe and a polynomial size of actual participants (for an arbitrary polynomial), and as we rely on cryptographic assumption, there is a non-zero-measure set of executions for which security is not guaranteed; this is inherent in the cryptographic setting where there is a negligible failure probability.
- Finally, their impossibility (3) considers a partially synchronous model, which requires that more than two-thirds of the parties are honest, even with a fixed and known party-set. As we consider the dishonest-majority setting, we assume synchronous communication, so this result does not apply to our setting.

Continuous consensus. Despite being new to the classical BA setting, in the continuous-consensus setting, several works have considered models with a similar flavor. (In fact, interest in such models was mostly sparked by Bitcoin and the so-called “permissionless” blockchains that fol-

lowed.) Although our work focuses on classical dishonest-majority broadcast (where the techniques are mostly different), for completeness we briefly survey these works and highlight the main differences. A number of works have performed novel analyses of blockchain protocols, and they mostly assume (implicitly or explicitly) that the universe of possible participants is polynomial. Among them, the first works that analyzed Bitcoin in the fixed-difficulty setting [GKL17, PSs17, BMTZ17] and the follow-ups that extended this analysis to the variable-difficulty setting [GKL24].

To our knowledge, the first work that introduced a model whereby participation fluctuates was the work by Pass and Shi on so-called *sleepy consensus* [PS17]. In their *sleepy model* for distributed protocols, parties can be either online or offline and can even switch between these two states. Already in that work, they noticed that classical consensus protocols do not work well in this model since the number of awake players may be significantly smaller than the total number of players. They then provided an instantiation of sleepy (continuous) consensus under strong assumptions, namely a PKI, common random strings, sub-exponentially secure collision-resistant hash functions, enhanced trapdoor permutations, and continuous honest majority—at any point in the execution, a majority of the awake players are honest. Their solutions and techniques, although for a more general (partial-)synchrony model, do not apply to dishonest majorities, even under their strong assumptions. The latency in the protocol of Pass and Shi [PS17] was later improved by Malkhi, Momose, and Ren [MR22, MMR23].

The sleepy model was employed in the design of the proof-of-stake consensus protocol Snow-white [DPS19] and later one generalized to the dynamic-availability setting [BGK⁺18] used to analyze the Ouroboros (Genesis) protocol. Both these works relied on the assumption of an honest majority of stake. Similar to the honest majority of hashing power used in Bitcoin, and more general blockchains following the proof-of-work paradigm, this is an alternative way to limit the adversary’s corruption capabilities: rather than assuming it controls a minority of parties, one makes parties’ pseudonyms take the role of parties in the execution, and makes the assumption that the number of corrupted pseudonyms is balanced by correlating the number of pseudonyms with the amount of resource held by their owners. (We refer to [AD15b, GKO⁺20, CDM25] for models that abstract this resource-restricting cryptography paradigm.) This is in contrast to our goal, which is to achieve broadcast in the cleanest possible setting—similar to classical BA—which makes no assumptions on the honesty of parties or on the balance of the resources they invest in the system. As a side note, our model also avoids a network assumptions which is crucial in [PS17]—namely that messages sent to a party who is away (sleeping), are delivered to this party once he joins back. In our setting, if a party is not around in some round, he simply misses the messages sent to him in that round and gets the guarantees (if any) of a late-joiner, based on the round he joins back.

Round complexity of (classical) BA. Dolev and Strong [DS83] first showed that any deterministic t -secure BA protocol requires at least $t + 1$ synchronized rounds. Ben-Or [Ben83] and Rabin [Rab83] showed that randomized protocols can help circumvent the above deterministic lower bound. Building on this idea, Feldman and Micali [FM97] presented an expected constant round randomized consensus protocol tolerating up to $t < n/3$ corruptions. This was later extended to the computational setting, where Katz and Koo [KK06] showed that assuming a PKI and existentially unforgeable signature expected constant round consensus can be constructed for up to $t < n/2$ corruptions.

Garay et al. [GKKO07] stretched the limits of feasibility of expected constant-round BA to $t = n/2 + O(1)$ corruptions (for Broadcast since consensus is impossible beyond honest majorities), and proved that this is asymptotically optimal: there does not exist any (randomized) broadcast

protocol that terminates in fewer than $2n/(n - t) - 1$ rounds for $n - t \geq 2$. Note that this, in particular, implies that if an adversary can corrupt arbitrary number of parties (up to $n - 2$), then $t + 1$ is a tight bound even for randomized broadcast protocols. Fitzi and Nielsen [FN09] proposed randomized protocols achieving expected $O(k)$ bounds when the number of Byzantine party is $n/2 + k$. More recently, Wan et al. [WXSD20] extended the feasibility of expected constant broadcast to allow for any constant fraction of corruptions. We note in passing, that all the above works are in the classical, known-participants setting.

Adaptive (adversary) security in BA. Hirt and Zikas [HZ10] showed that adaptive security makes BA feasibility far trickier than previously thought. In particular the classical, property-based definition of broadcast does not imply the simulation-based definition for broadcast functionality, so-called Megaphone functionality [CLOS02, GL05] if an adversary can adaptively corrupt parties and in the non-atomic multisend model (i.e., when a party might get corrupted in the middle of sending its messages to the network). The authors then defined a weaker functionality to capture such attacks as the *corruption-unfair* broadcast functionality. Cohen *et al.* [CGZ23] extended the impossibility result on adaptively secure broadcast to a stronger property-based broadcast definition, that required what they termed corruption fairness: in a nutshell, the adversary cannot decide whether or not to corrupt a party and use it to bias the protocol’s output based on this parties’ input before corruption. As they showed, circumventing the impossibility of this definition required strong assumptions like atomic multisend (an approach taken also in [WXDS20]) or time-lock puzzles (TLPs). To avoid extra complications that result in the adaptive security setting, in this work we will consider the classical definition of broadcast, without corruption fairness. Again, all the above works are in the classical, known participants setting.

Organization of the paper. In Section 2 we lay out the details of the unknowing participants (UP) model and define Broadcast in the UP model. In Section 3 we construct an active-parties set agreement (APSA) protocol, and in Section 4 an Interactive Consistency protocol (UPIC). These building blocks are combined into a Broadcast protocol in Section 5. Our lower bounds are proven in Section 6. Finally, in Section 7 we treat the case of late-joining parties.

2 Modeling Broadcast with Unknown Participants

We start off by describing the unknown-participants model, and in Section 2.2 we define the problem of Broadcast with unknown participants.

2.1 The Unknown-Participants Model

We begin by formally defining the *unknown-participants* (UP) model. As it will become evident from our discussion, the model itself has several intriguing details. As such, although the focus and technical substance of this paper is on protocols and impossibility results, we view the introduction of this new model as having its own merits.

The universe. In standard BA, parties have complete knowledge of the number and identities of all other parties in the network. Instead, in the *unknown-participants* (UP) model, we aim to achieve BA among parties who have no *a priori* knowledge about the identities or (even an estimate) about the number of other potential protocol participants. We will assume that each

party is identified by a unique string, which is unknown to the protocol, and/or its public key (as discussed below).

A caveat is that because parties need to be polynomial in the security parameter κ (otherwise they will be able to forge signatures) and at the same time be able to at least read the public key of other parties, the size of these strings must also be polynomial in κ . To capture this we consider a (potentially exponentially large) set of parties $\mathcal{U}$, which we refer to as the *universe*, where for simplicity we will assume that each of these parties is associated a unique party ID from $\{0, 1\}^{\text{poly}(\kappa)}$. Furthermore, to keep the protocol description simple we will use a party’s name and ID interchangeably; e.g., whenever we make a statement about a party $p \in \mathcal{U}$, we mean the party with the corresponding ID. Importantly, no party (except for the adversary, discussed below) has information about which, or how many, other parties are actually participating in an execution of the BA protocol with them. Note that because the parties are assumed to be probabilistic polynomial-time (PPT) in κ , the set of parties that are actually participating in the protocol (which we will call *active* or *activated parties*) will be a small subset of $\mathcal{U}$.

The environment and the adversary. To avoid over-complicating our treatment, we will stick to standard property-based definitions of the problem(s) (see, e.g., [CGZ23]). However, in order to capture the asymmetry of knowledge mentioned above while at the same time be able to make as general statements as possible, it will be useful to borrow some ideas from simulation-based security. In particular, we will consider a PPT Interactive Turing Machine (ITM), which we refer to as the *environment* $\mathcal{Z}$, that can activate parties from $\mathcal{U}$ at its discretion. We will consider two types of environments with respect to when they perform their party activation. A *static(-activations)* environment $\mathcal{Z}$ will activate all relevant parties from $\mathcal{U}$ at the beginning of the protocol execution, whereas a *dynamic(-activations)* environment may activate more parties during the protocol execution. We emphasize that since we assume the IDs are encoded in binary, even though the number of potential participants $\mathcal{U}$ can be exponential, the environment can still only activate a polynomially large subset.

In addition to the environment, we will assume an adversary $\mathcal{A}$, also a PPT ITM, that gets to corrupt activated parties and control them in a malicious (Byzantine) manner. Again, we will consider both standard flavors of adversaries, namely, *static* (where all corruptions are made before the protocol starts) and *adaptive* (where corruptions can occur during the protocol execution).

We stress that a party is considered *activated* only after receiving a special input from its environment. This means that parties cannot be activated by the adversary, by for example receiving a protocol message from a corrupted party. If the adversary attempts to send a message to a not-yet-activated party, this message is ignored. This separation of roles (between activating and corrupting) will be useful in our definitions and theorem statements. We remark that since the environment can activate any polynomial (in κ) number of parties, allowing any PPT adversary ensures that there are adversaries able to corrupt any number (even a majority) of parties. In fact, as we will discuss, such corrupted-majority adversaries are what makes the design of protocols particularly challenging.

To make the strongest possible statements, our negative results will hold for a static adversary and environment (this is the least adversarial scenario and will imply all others). For our protocols, wherever we state that the protocol can tolerate an adaptive adversary, we mean it as in the standard formulation in the MPC literature (see [CGZ23]) where the adversary can corrupt a party right after the party sends its messages to some of the parties, but before it completed sending to all parties.⁴ This is also the standard model for capturing adaptive corruptions in the MPC literature

⁴The ability of the adversary to corrupt a party in such a manner has also been referred to as “strongly rushing,”

(e.g., [CFGN96, Can00, Can01, CCL15]). We note, however, that as discussed in Section 1.2, we do not consider *corruption fairness* in our definitions, which makes them compatible with both this strong adaptive and weaker adaptive corruption models considered in the literature.

Active parties and honest parties in an execution. A caveat of the unknown-participants model just described is that the number and identities of activated/inactive and honest/corrupted parties depend on both the adversary and the environment. Hence, in order to make meaningful statements about the security properties of BA we need to refer to protocol *executions*, rather than to the protocol itself. This is a delicate point where the UP model deviates from the standard model, where only the adversary’s corruption capabilities need to be considered.

Driven by the above discussion we next introduce some notation and terminology that will be useful in our definitions and theorem statements. We start with the simple case of a static environment $\mathcal{Z}$ and adversary $\mathcal{A}$. Let $\mathcal{E}$ denote (the transcript⁵ of) an execution of a protocol in the UP model with such a static environment and adversary. We will denote by $\mathcal{P}_{\mathcal{E}} \subseteq \mathcal{U}$ the set of all parties that are activated by the environment, and by $\mathcal{H}_{\mathcal{E}} \subseteq \mathcal{P}_{\mathcal{E}}$ the set of all activated parties who are not corrupted by $\mathcal{A}$.

The situation becomes more involved once we consider dynamic environments and adaptive adversaries. To handle this case, we introduce the following, parameterized, version of the above notation. Let $\mathcal{E}$ be an execution with such an adaptive adversary $\mathcal{A}$ and dynamic environment $\mathcal{Z}$. For every $r \in \mathbb{N}^+$, we denote by $\mathcal{P}_{\mathcal{E}}^r \subseteq \mathcal{U}$ the set of parties that have been activated any round r' with $r' \leq r$; let $R_T \in \mathbb{N}^+$ be the (first) round where $\mathcal{Z}$ stops activating new parties (in slight abuse of terminology, we might refer to this round as the round where $\mathcal{Z}$ *terminates*), then for every $r \geq R_T$, denote $\mathcal{P}_{\mathcal{E}}^r = \mathcal{P}_{\mathcal{E}}^{R_T}$. Note that since the environment is polynomial in κ so is R_T . Similarly, we denote by $\mathcal{H}_{\mathcal{E}}^r \subseteq \mathcal{P}_{\mathcal{E}}^r$ the set of all parties in $\mathcal{P}_{\mathcal{E}}^r$ who are not corrupted by $\mathcal{A}$ at the end of round r .

It is worth noticing here that our formulation is different from the so-called *sleepy* model of consensus [PS17, DPS19] (a.k.a. *dynamic availability* [BGK⁺18]), where parties may join *and/or* leave at any point in the protocol. In our formulation, a party that joins (i.e., becomes active) remains active but might get corrupted by the adversary. We observe, however, that one can easily “project” the dynamic availability model onto ours, but this would be equivalent to UP where parties that leave in the middle of a protocol execution are treated as corrupted and lose their respective security guarantees; for example, if the sender of a broadcast drops out during the protocol, then there is no validity requirement on its input. We choose this modeling approach as we find it more appropriate for our setting—why should the sender in the above example get validity guaranteed?—while not adding unnecessary complications to the presentation of our results.

Network and synchrony. We will assume synchronous communication, where the protocol proceeds in rounds. As opposed to common assumptions in standard Byzantine agreement literature, our protocols do not require point-to-point communication between the parties (moreover, pairwise channels do not even make sense in the UP setting). Instead, we consider an abstract communication model in which once a party sends a message, it is guaranteed to arrive to all parties that are active in the same round (as in the *multicast network* from [BMTZ17, ACD⁺19]). The adversary can send different messages to different honest parties in each round. We remark that this abstraction is made to make the protocol description clear. In the motivating scenario of the UP model—where parties are unaware of other parties—this can be achieved by means of network assumptions which

see, e.g., [ADD⁺19, ACD⁺19, WXDS20].

⁵Such a transcript consists of all the inputs, outputs, randomness, setup, and incoming and outgoing messages of all parties, the adversary, and the environment.

are standard in the blockchain literature: parties use “gossiping” over an incomplete-but-connected synchronous network to send their messages, where a protocol-round lasts as much as the upper bound to the time/network-rounds it takes for a message to be delivered (which is the length of the longest honest path in the underlying communication graph).

Ad-hoc PKI setup. Classical impossibility results in the standard (known-participants) model [LSP82, FLM86, Bor96] imply that UP Broadcast is impossible without restricting the adversary’s power in some way. As we do not make any assumptions about on balanced resource-allocation (as common in the blockchain literature), we use the traditional approach of using digital signatures and some form of a public-key infrastructure (PKI). However, extra caution is needed to model such a PKI in our setting, as the PPT parties (as well as the adversary and the environment) would not be able to store the (exponentially many) public keys such a PKI for $\mathcal{U}$ would require.

To resolve this, we draw inspiration from how signature-based cryptography scales to massive networks like the Internet. In particular, rather than an exponentially large PKI, we assume the existence of an incorruptible *certification authority* (CA), holding a pair of keys $(\text{sk}_{\text{CA}}, \text{vk}_{\text{CA}})$ for a signature scheme, and whose public verification key vk_{CA} is known to the protocol. Once a party p is activated for the first time, p generates its own signature keys $(\text{sk}_p, \text{vk}_p)$ and sends vk_p to the CA, which in turns signs vk_p together with the identity of p to establish a certificate $\text{cert}_p \leftarrow \text{Sign}_{\text{sk}_{\text{CA}}}((\text{id}_p, \text{vk}_p))$ and sends cert_p to p .

Importantly, this CA is never used during the protocol execution other than issuing certificates. Instead, whenever a party p sends a signed message $\sigma_p \leftarrow \text{Sign}_{\text{sk}_p}(m)$ to some party q , party p attaches to this message its certified public key $(\text{id}_p, \text{vk}_p, \text{cert}_p)$.⁶ To verify the corresponding signature σ_p , party q first verifies that $\text{Verify}_{\text{vk}_{\text{CA}}}((\text{id}_p, \text{vk}_p), \text{cert}_p) = 1$, and then verifies that $\text{Verify}_{\text{vk}_p}(m, \sigma_p) = 1$. We will implicitly assume that this CA-credentials-based verification mechanism is part of any signature exchanged in this work.

We will abstract digital signatures using the modular approach of Canetti’s UC framework [Can01] and will use an ideal signature functionality $\mathcal{F}_{\text{sig}}$ [Can04, GKZ10]; specifically, we will use the recent ideal signature functionality from [CDL⁺24], which ensures idealized signature services without errors. This enables proving *perfect* security for protocols that use signatures in a black-box way (in the spirit of Dolev-Strong) and later, using the UC composition theorem, obtain computational security when instantiating $\mathcal{F}_{\text{sig}}$ with *any* signature scheme that is correct, existentially unforgeability under chosen-message attack (EUF-CMA), consistent (meaning that even with maliciously generated keys, a message-signature pair verifies in a consistent way except for negligible probability), and linear time (meaning that the signing complexity is linear in the message length). For the sake of brevity, whenever we refer in the paper to EUF-CMA signatures, we mean a signature scheme that is correct, EUF-CMA, consistent, and linear time.

This approach also enables the definition of a *deterministic* protocol that uses digital signatures, i.e., a protocol that is deterministic (in the sense that parties do not use their random tapes) and *perfectly secure* in the $\mathcal{F}_{\text{sig}}$ -model, and where $\mathcal{F}_{\text{sig}}$ is instantiated by a computationally secure signature scheme.

2.2 Byzantine Broadcast in the UP Model

We next proceed to define appropriate versions of the classical Byzantine broadcast problem [PSL80, LSP82], which “lift” the classical (i.e., known participants) definitions to the UP model.

⁶In fact, each party p would only need to do it for the first signature it sends to q , but for simplicity we will assume that all signed messages are accompanied by the certified public key.

As a warm-up, we begin with the version that assumes a static environment and a static adversary—i.e., the set of active parties is fixed at the beginning of the protocol and so is the set of corrupted parties amongst them. This simpler case will be useful for our impossibility results, and already demonstrates the challenges, novelties, and idiosyncrasies of the UP model. In particular, the two key ways in which the definition deviates from the standard one are: (1) Unlike in the known-participants model, the definition of (UP-)Broadcast cannot simply define properties of the broadcast protocol. The reason is that these properties depend on the set of active parties, which is not defined (or known) in the protocol specification. (2) We will allow the protocol to operate even with a sender who is *inactive*, and when it does, we will require that the output of every party will be $\perp$. We note that this seemingly simple adaptation is in fact a key source of complication in the protocol construction.

Definition 2.1 (UP Broadcast: static). *Let $\kappa \in \mathbb{N}$ be a security parameter, let $\mathcal{U}$ be a (possibly exponential) set of parties, and let $s \in \mathcal{U}$. A protocol π is a **statically secure UP Broadcast protocol with sender s** if for every static PPT environment $\mathcal{Z}$, every static PPT adversary $\mathcal{A}$, and every input message $m_s \in \{0, 1\}^\ell$ with $\ell = O(\kappa)$, with overwhelming probability, an execution $\mathcal{E}$ of π with sender input m_s satisfies the following properties:*

- **Validity:** *Every honest party p terminates with output $m_p \in \{0, 1\}^\ell \cup \{\perp\}$, where $m_p = m_s$ if $s \in \mathcal{H}_{\mathcal{E}}$ and $m_p = \perp$ if $s \notin \mathcal{P}_{\mathcal{E}}$.*
- **Consistency:** *For all honest parties p and q which output m_p and m_q , respectively, it holds that $m_p = m_q$.*
- **Termination:** *All honest parties terminate and output a value in polynomially (in κ) many rounds.*

Having defined the above static version, we are now ready to define our adaptively secure version. There are several more challenges here, which are related to which parties should receive guarantees (e.g., initially active vs. late joining parties), and even how these guarantees can be formalized. Consider for example the adaptation of the *validity* property to the adaptive setting: If the sender is honest at the end of the protocol, then every honest party outputs the sender’s input. There are several questions that this description leaves open. For example, (1) What if the sender did not join the protocol at the beginning? and (2) What constitutes “the end” of the protocol? Indeed, the environment might activate parties at any point so the “end” must be defined by means of looking at the parties’ outputs.

There are actually several ways to answer the above questions, with each giving guarantees of different strength. Here we choose what we think to be the more direct adaptation of the static to the adaptive definition, starting with Question 2: Since protocols might not have simultaneous termination, the “end” of the protocol is defined with respect to each honest party as the round in which this party terminates and outputs. Then, for Question 1, a sender is guaranteed to correctly broadcast its input if it is active at the beginning and honest at the time where the last honest party who has also been around from the beginning terminates with an output. The formal definition follows:

Definition 2.2 (UP Broadcast: dynamic). *Let $\kappa \in \mathbb{N}$ be a security parameter, let $\mathcal{U}$ be a (possibly exponential) set of parties, and let $s \in \mathcal{U}$. A protocol π is an **adaptively secure UP Broadcast protocol with sender s** , if for every dynamic PPT environment $\mathcal{Z}$, every adaptive PPT adversary $\mathcal{A}$, and every input message $m_s \in \{0, 1\}^\ell$ with $\ell = O(\kappa)$, with overwhelming probability, an execution $\mathcal{E}$ of π with sender input m_s in the presence of $\mathcal{Z}$ and $\mathcal{A}$ satisfies the following properties:*

- **Validity:** Let r be a round number, and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^r$ be a party who terminates in round r with output m . Then $m \in \{0, 1\}^\ell \cup \{\perp\}$, where $m = m_s$ if $s \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^r$, and $m = \perp$ if $s \notin \mathcal{P}_\mathcal{E}^1$.
- **Consistency:** Let r_p and r_q be round numbers, and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_p}$ and $q \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_q}$ be two parties that terminate in rounds r_p and r_q with outputs m_p and m_q , respectively. Then $m_p = m_q$.
- **Termination:** All parties in $\mathcal{P}_\mathcal{E}^1$ either terminate in polynomially (in κ) many rounds, or they get corrupted before they terminate.

Note that the above definition does not give any guarantees to parties who join the execution of the broadcast protocol after it started. In Section 7, we provide refinements of the definition (and corresponding feasibility results) that offer guarantees also for late-joining parties (who remain honest).

3 Active-Parties Set Agreement Protocol

In this section, we will provide an *active-parties set agreement* (APSA) protocol. The protocol tolerates any number of corruptions in the unknown-participants model. In Section 3.1, we show how to adapt the idea of Dolev-Strong broadcast (DS-Broadcast) to our model and give a high-level description of the protocol, in Section 3.2 we give a formal description, and in Section 3.3 a security proof.

We begin by formally describing the properties required from an APSA protocol. As discussed in the introduction, our description uses parties and their identifiers interchangeably. So what APSA outputs is in fact an agreed upon set of party identifiers. Similar to UP Broadcast (Section 2.2), we first provide the definition for static environments and adversaries, and then extend to the dynamic/adaptive case.

Definition 3.1 (APSA: static). *Let $\kappa \in \mathbb{N}$ be a security parameter and let $\mathcal{U}$ be a (possibly exponential) set of parties. A protocol π is a **statically secure active-parties set agreement (APSA) protocol** in the UP setting, if for every static PPT environment $\mathcal{Z}$ and every static PPT adversary $\mathcal{A}$, with overwhelming probability, an execution $\mathcal{E}$ of π in the presence of $\mathcal{Z}$ and $\mathcal{A}$ satisfies the following properties:*

- **Validity:** Every honest party p outputs a set S , such that $\mathcal{H}_\mathcal{E} \subseteq S \subseteq \mathcal{P}_\mathcal{E}$.
- **Consistency:** For all honest parties p and q which output S_p and S_q , respectively, $S_p = S_q$.
- **Termination:** All honest parties terminate with an output in polynomially (in κ) many rounds.

Definition 3.2 (APSA: dynamic). *Let $\kappa \in \mathbb{N}$ be a security parameter and let $\mathcal{U}$ be a (possibly exponential) set of parties. A protocol π is an **adaptively secure active-parties set agreement protocol** in the UP setting, if for every dynamic PPT environment $\mathcal{Z}$ and every adaptive PPT adversary $\mathcal{A}$, with overwhelming probability, an execution $\mathcal{E}$ of π in the presence of $\mathcal{Z}$ and $\mathcal{A}$, satisfies the following properties:*

- **Validity:** Let r be a round number, and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^r$ be a party that terminates in round r with output set S . Then $\mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^r \subseteq S \subseteq \mathcal{P}_\mathcal{E}^r$.
- **Consistency:** Let r_p and r_q be round numbers, and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_p}$ and $q \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_q}$ be two parties that terminate in rounds r_p and r_q with outputs S_p and S_q , respectively. Then $S_p = S_q$.

- **Termination:** All parties in $\mathcal{P}_{\mathcal{E}}^1$ either terminate in polynomially (in κ) many rounds or they get corrupted before they terminate.

Notice that in APSA the parties have no input and that the output guarantees only concern parties which are activated at the beginning of the execution and remain honest until they terminate.

3.1 Adapting Dolev-Strong to the UP Model

In order to achieve agreement on a set of active parties, we use a protocol structure similar to DS-Broadcast [DS83] adapted as outlined in Section 1.1. In particular, instead of transferring votes/signed-certificate-chains for the input of a designated sender, the parties shall transfer votes for the participation of other parties along with some proof that these parties have actively participated at some point during the execution of the protocol. Note that the only way some party $p \in \mathcal{U}$ should get convinced about the actual participation of some other party $u \in \mathcal{U}$ is by observing u 's signature (along with the certificate from the CA on u 's id and verification key).

The protocol proceeds as follows: In the initial round ($r = 1$), every activated party $p \in \mathcal{U}$ signs id_p and sends it to all active parties. From that point and on and in every round $r > 1$, a party p accepts some other party $u \in \mathcal{U}$ as active, only if it has not accepted u by round $r - 1$ and observes signatures on id_u from at least $r - 1$ distinct parties including u , out of which everyone, except from u , was accepted by p as active by round $r - 1$. Note that for p to accept some new party in round r , it must have accepted at least $r - 2$ parties (apart from itself) by round $r - 1$. So, if p observes the number of its so-far accepted parties to be at most $r - 1$, it knows it will not accept any new party in the rounds that follow, and thus, it can safely terminate. Otherwise, the party needs to convince the rest of the so-far honest parties about the participation of every party it newly accepted in that round r . Hence, p collects all the signatures it observed in round r in support of the participation of u , adds its own signature on id_u , and sends this new batch of signatures to all active parties.

Here, one can ask: How can p be certain that none of the other uncorrupted parties has terminated at some round prior to $r + 1$ without accepting u (which would violate the *consistency* property of the APSA)? Intuitively, the reason follows since every party in $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^1$ can convince all parties in $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^2$ about its participation by round 2 and make them proceed to round 3. Similarly, this implies that every party in $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ can also convince every party in $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{r+1}$ in round $r + 1$ about the participation of every party it has accepted by round $r - 1$ including itself. This, in turns, makes all of them proceed to round $r + 2$. We formally prove the above intuitive argument in Section 3.3 where we also discuss the protocol's round complexity.

3.2 The APSA Protocol

We next provide the formal description of our APSA protocol. We start by introducing the notion of *valid batches of signatures* and *well-formed messages*. Since the output of APSA actually consists of identifiers of parties, it is worth recalling that we assume that each party has a unique ID id_p by which it has registered its verification key vk_p with the certification authority CA. Recall also that without loss of generality we assume that every signature includes the signer's ID, its verification key, and CA's certificate, i.e., when we say a party sends $\text{Sign}_p(m)$, the message contains $(\text{id}_p, \text{vk}_p, \text{cert}_p, m, \sigma_p(m))$, where $\sigma_p(m)$ is the actual signature on m with p 's signing key.

Valid batches of signatures. Let $S_p^r \subseteq \mathcal{U}$ be the set of parties that some active party $p \in \mathcal{U}$ has accepted by round $r \geq 1$ and let B be a batch of signatures on id_u , for some $u \in \mathcal{U}$. Then, we will call B a (p, r) -*valid batch of signatures* for u if B consists of signatures on id_u from at least $r - 1$

distinct parties including u , such that at least $r - 2$ of them are included in S_p^{r-1} . More formally, let $B = \{\text{Sign}_v(\text{id}_u)\}_{v \in V}$, for some $V \subseteq \mathcal{U}$, then B is a (p, r) -valid batch of signatures if $u \in V$ and $|V \cap S_p^{r-1} \setminus \{u\}| \geq r - 2$.

Well-formed messages. As we mentioned earlier, in every round $r \geq 1$, we would like all honest parties (which have decided to proceed to the next round) to send batches of signatures in support of the participation of every party they have newly accepted. So, since we want to limit the lengths of messages generated by corrupted parties, every activated party $p \in \mathcal{U}$ shall include at most one batch of signatures for each newly accepted party in the message it sends at the end of every round r . In particular, we will call such a message M_p^r *well-formed* if $M_p^r = \{B_i\}_{i=1}^k$, for some $k \geq 1$, where for every $i \in [k]$, the set B_i is a batch of signatures in support of party $u_i \in \mathcal{U}$; that is $B_i = \{\text{Sign}_v(\text{id}_{u_i})\}_{v \in V_i}$, for some $V_i \subseteq \mathcal{U}$ with $u_i \in V_i$. Further, the parties $u_1, \dots, u_k$ are pairwise distinct. In the following protocol, honest parties will only process *well-formed* messages and discard the rest.

We give a formal description of our protocol in Figure 1.

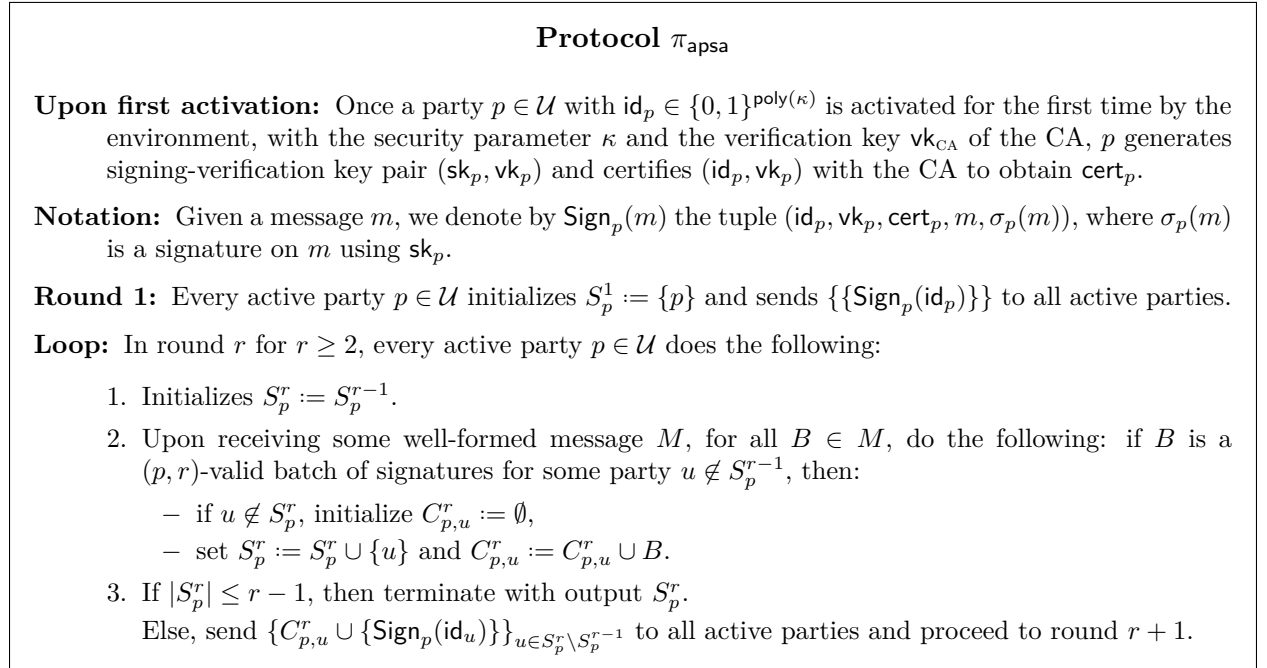


Figure 1: Active-Parties Set Agreement Protocol.

3.3 APSA Protocol Analysis

We proceed to formally prove that the protocol π_{apsa} satisfies Definition 3.2.

Theorem 3.3. *Assuming the existence of EUF-CMA signatures, Protocol π_{apsa} (Figure 1) is an adaptively secure APSA protocol (according to Definition 3.2) in the CA-model.*

Moreover, in every execution $\mathcal{E}$ of π_{apsa} , all parties in $\mathcal{P}_{\mathcal{E}}^1$ that terminate uncorrupted, terminate simultaneously in at most $|\mathcal{P}_{\mathcal{E}}^{R_T}|$ rounds, where R_T is the last round in which the environment activates new parties.

Proof. Let $\mathcal{Z}$ be a dynamic PPT environment and let $\mathcal{A}$ be an adaptive PPT adversary. Let $\mathcal{E}$ be an execution of π_{apsa} under $\mathcal{Z}$ and $\mathcal{A}$. We will prove that *validity*, *consistency*, and *termination*

hold perfectly in $\mathcal{E}$ when signatures are modeled ideally in the $\mathcal{F}_{\text{sig}}$ -hybrid model, which implies that the properties hold with overwhelming probability over the randomness of $\mathcal{Z}$, $\mathcal{A}$, the activated parties, and the CA, once $\mathcal{F}_{\text{sig}}$ is instantiated with EUF-CMA signatures (note that active parties and the CA use their randomness only for the signature scheme).

During the proof, we will say that some party $p \in \mathcal{U}$ *accepts* some party $u \in \mathcal{U}$ in round r , if r is the first round in which u is in S_p^r , i.e., $r = \min\{r' \in \mathbb{N}^+ : u \in S_p^{r'}\}$. We will prove validity in Lemma 3.4, consistency in Lemma 3.5, and simultaneous termination in Lemma 3.7.

Lemma 3.4 (Validity). *Let r be a round, and let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ be a party that terminates in round r with output set S_p . Then, $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r \subseteq S_p \subseteq \mathcal{P}_{\mathcal{E}}^r$.*

Proof. First, observe that for any $u \in S_p \setminus \{p\}$, party p must have received a (p, r') -valid batch of signatures in support of u in some round $r' \leq r$; denote this batch by B . Thus, by the definition of valid batches of signatures, $\text{Sign}_u(\text{id}_u) \in B$. Hence, $u \in \mathcal{P}_{\mathcal{E}}^r$ and thus, we have that $S_p \subseteq \mathcal{P}_{\mathcal{E}}^r$.

It remains to show that $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r \subseteq S_p$. Notice that $p \in S_p^1$ and by construction $p \in S_p$. Moreover, every $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$, with $q \neq p$, sends $\{\{\text{Sign}_q(\text{id}_q)\}\}$ at the end of round 1 and thus, p receives it in round 2. Hence, p accepts q , since $q \notin S_p^1$ and $\{\text{Sign}_q(\text{id}_q)\}$ is a $(p, 2)$ -valid batch of signatures for q . So, $q \in S_p^2 \subseteq S_p$ by construction, which completes the proof. $\square$

Notice that for the *consistency* property to be violated, there must exist two parties p and q that terminate uncorrupted, and some other party u such that u is included in the output of p but not in the output of q . Hence, in order to prove that the *consistency* property is satisfied, it suffices to show that whenever some party $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ accepts some party u in round r , then every party $q \in \mathcal{P}_{\mathcal{E}}^1$ that terminates uncorrupted also accepts u before terminating. We will prove this statement using induction on the number of rounds.

Lemma 3.5 (Consistency). *Let r be a round number, and let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ be a party which accepts some party u in round r . Then, every party $q \in \mathcal{P}_{\mathcal{E}}^1$ that terminates uncorrupted accepts u by round $r + 1$ before terminating.*

Proof. First, we will show that the lemma holds for the base cases $r = 1$ and $r = 2$. In round $r = 1$, every party $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^1$ only accepts itself and sends $\{\{\text{Sign}_p(\text{id}_p)\}\}$ to all active parties. Hence, every party $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^2$, with $q \neq p$, receives p 's message in round 2 and accepts p , since $p \notin S_q^1 = \{q\}$ and $\{\text{Sign}_p(\text{id}_p)\}$ is a $(q, 2)$ -valid batch of signatures for p .

For round $r = 2$, let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^2$. Note that p accepts a party u in round 2 if $u \neq p$ and p receives a $(p, 2)$ -valid batch of signatures for u of the form $\{\{\text{Sign}_u(\text{id}_u)\}\}$ in round 2. In this case, p will send the message $\{\{\text{Sign}_u(\text{id}_u)\}, \{\text{Sign}_p(\text{id}_u)\}\}$ to all active parties. Now consider a party $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^2$. If $q = u$ then q accepted u in round 1, and if $q = p$ then q accepted u in round 2. If $q \notin \{p, u\}$, then either q accepted u in round 2, or q receives p 's message in round 3 and accepts u , since $u \notin S_q^2$ (since q did not accept u yet) and $\{\{\text{Sign}_u(\text{id}_u)\}, \{\text{Sign}_p(\text{id}_u)\}\}$ is a $(q, 3)$ -valid batch of signatures for u .

Now let $r \geq 3$ and assume that the lemma holds for every round $r' < r$. Consider a party $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ and assume that p accepts u in round r . This means that p receives a (p, r) -valid batch of signatures for u in round r ; denote this batch of signatures by $B = \{\text{Sign}_v(\text{id}_u)\}_{v \in V}$ for some $V \subseteq \mathcal{U}$, and observes that $u \notin S_p^{r-1}$ and that $|V \cap S_p^{r-1}| \geq r - 1$. By definition of a valid batch of signatures, there exists a set of parties $W \subseteq V \cap S_p^{r-1}$ with $|W| \geq r - 2$, such that p has accepted every party of W by round $r - 2$. By the perfect guarantees of $\mathcal{F}_{\text{sig}}$, it holds that $p \notin V$ and thus $p \notin W$ (since $p \in V$ would imply that p 's signature on id_p was forged prior to r).

Let $q \in \mathcal{P}_{\mathcal{E}}^1$ be some party that terminates uncorrupted with $q \neq p$. By the induction hypothesis, since p accepts all parties in W by round $r - 2$, it holds that q accepts all parties of W by round $r - 1$. Now, consider two cases:

If q has accepted u by round r , then the proof follows.

Otherwise, i.e., if q has not accepted u by round r , then, in particular, $q \notin W$ (as all honest parties in W accepted u before round r). Let $r' \leq r$ be the last round in which q accepted some party of W (note that $r' \geq 2$ since $q \notin W$). Note that q accepts q in round 1 and, q accepts p in round 2 (since $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^1$), which means that $W \cup \{p, q\} \subseteq S_q^{r'}$ in round r' . Therefore, for every round $r'' \in [r', r]$ it holds that

$$|S_q^{r''}| \geq |S_q^{r'}| \geq |W \cup \{p, q\}| \geq r > r'' - 1,$$

where the first inequality holds because $r'' \geq r'$ hence $S_q^{r'} \subseteq S_q^{r''}$, the second because $W \cup \{p, q\} \subseteq S_q^{r'}$, the third because $|W| \geq r - 2$ and $p, q \notin W$, and the fourth because $r \geq r''$. Hence, in every round $r'' \in [r', r]$, it holds that $|S_q^{r''}| > r'' - 1$, and thus q participates in round $r + 1$. Since p is honest and accepted u in round r , it holds that p sends a well-formed message in round r that contains a batch of signatures for u ; denote this batch of signatures by $B' = \{\text{Sign}_v(\text{id}_u)\}_{v \in V'}$ with $V \cup \{u\} \subseteq V'$. Therefore, q receives B' in round $r + 1$ and observes that $u \in V' \setminus S_q^r$ and that $|V' \cap S_q^r| \geq |W \cup \{p\}| \geq r - 1$. Thus, q accepts u in round $r + 1$. $\square$

Finally, we will prove not only that *termination* holds, but also that in $\mathcal{E}$ all uncorrupted parties (activated from the beginning of the execution) terminate simultaneously.

Lemma 3.6 (Termination). *Let $R_T \in \mathbb{N}$ be the round number when $\mathcal{Z}$ terminates, then all parties in $\mathcal{P}_{\mathcal{E}}^1$ either terminate in at most $|\mathcal{P}_{\mathcal{E}}^{R_T}|$ rounds or they get corrupted before they terminate.*

Proof. Let $\bar{R} = |\mathcal{P}_{\mathcal{E}}^{R_T}|$ and let $p \in \mathcal{P}_{\mathcal{E}}^1$ be a party that neither gets corrupted (before terminating) by round $\bar{R} + 1$ nor terminates by round $\bar{R}$. Then $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{\bar{R}+1}$ and for every $u \in S_p^{\bar{R}+1}$, p must have received $\text{Sign}_u(\text{id}_u)$. Thus, we have that $S_p^{\bar{R}+1} \subseteq \mathcal{P}_{\mathcal{E}}^{\bar{R}} \subseteq \mathcal{P}_{\mathcal{E}}^{R_T}$ and hence, in round $\bar{R} + 1$, p observes $|S_p^{\bar{R}+1}| \leq \bar{R}$. Therefore, p terminates in round $\bar{R} + 1$. $\square$

Lemma 3.7 (Simultaneous Termination). *All parties in $\mathcal{P}_{\mathcal{E}}^1$ that terminate uncorrupted, terminate simultaneously.*

Proof. Let S_p be the output of a party $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ that terminates in round r , then p observes $S_p = S_p^r$ and $|S_p^r| \leq r - 1$ in round r . If we assume that $|S_p^r| < r - 1$, then we obtain that $|S_p^{r-1}| \leq |S_p^r| \leq r - 2$, meaning that p would have terminated by round $r - 1$, which is a contradiction. So, for the round r it holds that $r - 1 = |S_p^r| = |S_p|$. By Lemma 3.5, all parties that terminate uncorrupted output the same set S_p and thus, terminate simultaneously in round $r = |S_p| + 1$. $\square$

By Lemmas 3.4, 3.5, 3.6, and 3.7 we conclude the proof of Theorem 3.3. $\square$

From APSA to UP Broadcast. Notice that the simultaneous termination of our APSA protocol constitutes the protocol sufficient to solve UP Broadcast. In particular, one can easily come up with a UP Broadcast protocol by running APSA to achieve agreement on a set of active parties and then, run Dolev-Strong on the agreed party set. However, this solution doubles the number of rounds required for termination. In contrast, in the next section we extend our APSA protocol into an *interactive consistency* protocol in the UP setting, which requires exactly the same number of rounds as the APSA protocol. Hence, we make use of this *interactive consistency* protocol to achieve UP Broadcast in the same number of rounds as APSA.

4 Interactive Consistency with Unknown Participants

In this section, we define and construct an *unknown-participants interactive consistency* (UPIC) protocol that tolerates any number of corruptions. We begin by defining the properties of UPIC. Next, in Section 4.1 we present the high-level overview of the construction, in Section 4.2 formally describe our protocol, and in Section 4.3, we prove its security.

We begin by providing a definition for the interactive consistency protocol in the UP model. In this problem, every active party $p \in \mathcal{U}$ is given an initial input value m_p and all parties that terminate uncorrupted shall agree on a common set of party-value pairs, so then each such pair (u, m) indicates that m has been decided as the input value of $u \in \mathcal{U}$. If u was activated at the beginning of the execution and remained honest until the time that p terminates, then m must be equal to m_u . Once again we provide the definition of the problem for both static and adaptive environments and adversaries.

Definition 4.1 (UPIC: static environments & adversaries). *Let $\kappa \in \mathbb{N}$ be a security parameter, let $\mathcal{U}$ be a (possibly exponential) set of parties, and let $\ell = O(\kappa)$. A protocol π is a **statically secure UP interactive consistency protocol**, if for every static PPT environment $\mathcal{Z}$, every static PPT adversary $\mathcal{A}$, and every set of input messages $\{m_p \in \{0, 1\}^\ell : p \in \mathcal{U}\}$, with overwhelming probability, an execution $\mathcal{E}$ of π in the presence of $\mathcal{Z}$ and $\mathcal{A}$, satisfies the following properties:*

- **Validity:** *Every party $p \in \mathcal{H}_{\mathcal{E}}$ outputs a set of party-value pairs $A_p = \{(q, m'_q)\}_{q \in S_p}$, such that*
 1. $\mathcal{H}_{\mathcal{E}} \subseteq S_p \subseteq \mathcal{P}_{\mathcal{E}}$.
 2. *For every $q \in S_p$, it holds that $m'_q \in \{0, 1\}^\ell \cup \{\perp\}$, and if $q \in \mathcal{H}_{\mathcal{E}}$ then $m'_q = m_q$.*
- **Consistency:** *If parties $p, q \in \mathcal{H}_{\mathcal{E}}$ output A_p and A_q , resp., then $A_p = A_q$.*
- **Termination:** *All honest parties terminate with an output in polynomially (in κ) many rounds.*

Definition 4.2 (UPIC: dynamic environments & adaptive adversaries). *Let $\kappa \in \mathbb{N}$ be a security parameter, let $\mathcal{U}$ be a (possibly exponential) set of parties, and let $\ell = O(\kappa)$. A protocol π is an **adaptively secure UP interactive consistency protocol**, if for every dynamic PPT environment $\mathcal{Z}$, every adaptive PPT adversary $\mathcal{A}$, and every set of input messages $\{m_p \in \{0, 1\}^\ell : p \in \mathcal{U}\}$, with overwhelming probability, an execution $\mathcal{E}$ of π in the presence of $\mathcal{Z}$ and $\mathcal{A}$, satisfies the following properties:*

- **Validity:** *Let r be a round number, and let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ be a party who terminates in round r with output $A_p = \{(q, m'_q)\}_{q \in S_p}$. Then,*
 1. $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r \subseteq S_p \subseteq \mathcal{P}_{\mathcal{E}}^r$.
 2. *For every $q \in S_p$, it holds that $m'_q \in \{0, 1\}^\ell \cup \{\perp\}$, and if $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ then $m'_q = m_q$.*
- **Consistency:** *Let r_p and r_q be round numbers, and let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{r_p}$ and $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{r_q}$ be two parties that terminate in rounds r_p and r_q with outputs A_p and A_q , respectively. Then $A_p = A_q$.*
- **Termination:** *All parties in $\mathcal{P}_{\mathcal{E}}^1$ either terminate in polynomially (in κ) many rounds or they get corrupted before they terminate.*

In the above definitions, the *consistency* property is analogous to the one of APSA (Section 3). The *validity* property requires that the output of the honest parties must be a set of party-value pairs, such that all pairs in the set contain distinct active parties and values from $\{0, 1\}^\ell \cup \{\perp\}$, including all input values of so-far honest parties activated from the beginning.

4.1 Building on the APSA Protocol

Once we have a protocol that achieves agreement on a set of active parties (which includes all honest parties), we can use it as the basis of an UPIC protocol.

Now, in round 1, every active party $p \in \mathcal{U}$, instead of signing id_p , signs $\text{id}_p || m_p$ and sends it to all active parties. So, all parties shall send batches of signatures in support of party-value pairs of the form $\text{Sign}_v(\text{id}_u || m)$, where $u, v \in \mathcal{U}$ and $m \in \{0, 1\}^\ell$. The acceptance condition for a party-value pair (u, m) , which is also the acceptance condition for the participation of u , is almost identical to the one for the participation of a party we had in our APSA protocol. That is, any party $p \in \mathcal{U}$ accepts some pair (u, m) (as well as the participation of u) in round $r > 1$, only if p has not accepted (u, m) by round $r - 1$ and p observes signatures on $\text{id}_u || m$ from at least $r - 1$ distinct parties including u , out of which everyone except for u , was accepted by p by round $r - 1$. Once some party observes the number of its so-far accepted parties to be at most $r - 1$, it can safely terminate, exactly as in our APSA protocol.

The difference now is that the output of every party p will be a set of party-value pairs, which will indicate the messages that p decides for each one of its accepted parties. In particular, for every party u that p has accepted, p will either decide the default value $\perp$, if it has accepted more than one $(u, \cdot)$ pairs throughout the execution, or some value $m \in \{0, 1\}^\ell$, if (u, m) is the only pair it has accepted up to that point with respect to u . In this way, the agreement on the parties' output is implied by a relay property very similar to the one we proved in Lemma 3.5 for our APSA protocol, which will now concern the acceptance of party-value pairs.

4.2 The UPIC Protocol

We proceed to give a formal description of our UPIC protocol which tolerates any number of corruptions in the unknown participants model. First, we have to extend the definitions of *valid batches of signatures* and *well-formed messages* that we gave in Section 3.2. A formal description of our protocol is given in Figure 2.

Valid batches of signatures. We extend the definition of *valid batches of signatures* to the case that the signatures of the batch are in support of a party-value pair. Let B be a batch of signatures on some party-value pair $(u, m) \in \mathcal{U} \times \{0, 1\}^\ell$, then we will call B a (p, r) -*valid batch of signatures*, if it consists of signatures on $\text{id}_u || m$ from at least $r - 1$ distinct parties including u , such that at least $r - 2$ of them are included in $S_p^{r-1} \setminus \{u\}$, i.e., let $B = \{\text{Sign}_v(\text{id}_u || m)\}_{v \in V}$, for some $V \subseteq \mathcal{U}$, then B is a (p, r) -*valid batch of signatures* if $u \in V$ and $|V \cap S_p^{r-1} \setminus \{u\}| \geq r - 2$.

Well-formed messages. In our UPIC protocol, the honest parties will have to send batches of signatures in support of the participation of their newly accepted party-value pairs, exactly as they did in our APSA protocol. To maintain APSA's efficiency, every party shall send at most one batch of signatures for each newly accepted pair. Let M_p^r be a message sent by some party $p \in \mathcal{U}$ sends at the end of round r , then M_p^r will be called *well-formed* if $M_p^r = \{B_j\}_{j=1}^k$, for some $k \geq 1$, where B_i are batches of signatures in support of distinct pairs (u_i, m_i) , such that for all $i \in [k]$, $B_i = \{\text{Sign}_v(\text{id}_{u_i} || m_i)\}_{v \in V_i}$, for some $V_i \subseteq \mathcal{U}$ with $u_i \in V_i$.

Output. In the last step in the protocol, the parties extract their output from the values they received thus far. For this purpose, we defined the following function **Out**. Let A_p^r be the set of party-value pairs that some party $p \in \mathcal{U}$ has accepted by round r . For every $u \in S_p^r$, let $A_{p,u}^r = \{m \in \{0, 1\}^\ell : (u, m) \in A_p^r\}$ be the set of values that p has accepted by round r with respect

to u . Then, if p decides to terminate in round r , it outputs the party-value pair for every party u for which $A_{p,u}^r$ contains exactly one value, and $\perp$ for every party u for which $A_{p,u}^r$ contains more than one value. That is,

$$\text{Out}(A_p^r) = \left\{ (u, m'_u) \right\}_{u \in \hat{S}_p^r} \cup \left\{ (u, \perp) \right\}_{u \in S_p^r \setminus \hat{S}_p^r},$$

where $\hat{S}_p^r = \left\{ u \in S_p^r : \exists m'_u \in \{0, 1\}^\ell \text{ such that } A_{p,u}^r = \{m'_u\} \right\}$.

Protocol π_{ic}

Upon first activation: Once a party $p \in \mathcal{U}$ with $\text{id}_p \in \{0, 1\}^{\text{poly}(\kappa)}$ is activated for the first time by the environment, with the security parameter κ , with the verification key vk_{CA} of the CA, and with input $m_p \in \{0, 1\}^\ell$, party p generates signing-verification key pair $(\text{sk}_p, \text{vk}_p)$ and certifies $(\text{id}_p, \text{vk}_p)$ with the CA to obtain cert_p .

Notation: Given a message m , we denote by $\text{Sign}_p(m)$ the tuple $(\text{id}_p, \text{vk}_p, \text{cert}_p, m, \sigma_p(m))$, where $\sigma_p(m)$ is a signature on m using sk_p .

Round 1: Every active party $p \in \mathcal{U}$ initializes $S_p^1 := \{p\}$ and $A_p^1 := \{(p, m_p)\}$ and sends $\{\{\text{Sign}_p(\text{id}_p || m_p)\}\}$ to all active parties.

Loop: In round r for $r \geq 2$, every active party $p \in \mathcal{U}$ does the following:

1. Initializes $S_p^r := S_p^{r-1}$ and $A_p^r := A_p^{r-1}$.
2. Upon receiving some well-formed message M , for all $B \in M$, do the following: if B is a (p, r) -valid batch of signatures for some pair $(u, m) \notin A_p^{r-1}$, then:
 - if $(u, m) \notin A_p^r$, initialize $C_{p,u,m}^r := \emptyset$,
 - set $S_p^r := S_p^r \cup \{u\}$, $A_p^r := A_p^r \cup \{(u, m)\}$, and $C_{p,u,m}^r := C_{p,u,m}^r \cup B$.
3. If $|S_p^r| \leq r - 1$, then terminate with output $\text{Out}(A_p^r)$; else, send to all active parties the set

$$\left\{ C_{p,u,m}^r \cup \{\text{Sign}_p(\text{id}_u || m)\} \right\}_{(u,m) \in A_p^r \setminus A_p^{r-1}}$$

and proceed to round $r + 1$.

Figure 2: UP Interactive Consistency Protocol.

Note that p stores in every round $r > 1$ all the signatures it has observed in support of every party-value pair $(u, m) \in A_p^r \setminus A_p^{r-1}$ in the $C_{p,u,m}^r$ sets. The way it constructs the message that it sends at the end of round r is the same as in our APSA protocol.

4.3 Interactive Consistency Protocol Analysis

We now turn to prove that π_{ic} satisfies Definition 4.2. Similarly to the proof of Theorem 3.3, we will say that some party $p \in \mathcal{U}$ *accepts* some pair (u, m) in round r , if $r = \min\{r' \in \mathbb{N}^+ : (u, m) \in A_p^{r'}\}$.

Notice that the termination condition of π_{ic} , as well as the handling of the sets of accepted parties, is exactly the same as in π_{apsa} (Figure 1). Further, the number of pairs accepted with respect to some particular party does not have any impact on the decision of the parties about the termination of the protocol, except for the acceptance of the party itself. Hence, all the properties regarding the acceptance of parties and the satisfaction of the termination condition that we proved for π_{apsa} (Lemmas 3.5, 3.6, and 3.7) will hold in our π_{ic} as well.

Theorem 4.3. *Assuming the existence of EUF-CMA signatures, Protocol π_{ic} (Figure 2) is an adaptively secure UPIC protocol (according to Definition 4.2) in the CA-model.*

Moreover, in every execution $\mathcal{E}$ of π_{ic} , all parties in $\mathcal{P}_{\mathcal{E}}^1$ that terminate uncorrupted, terminate simultaneously in at most $|\mathcal{P}_{\mathcal{E}}^{R_T}|$ rounds, where R_T is the last round in which the environment activates new parties.

Proof. Let $\mathcal{Z}$ be a dynamic PPT environment and let $\mathcal{A}$ be an adaptive PPT adversary. Let $\mathcal{E}$ be an execution of π_{ic} under $\mathcal{Z}$ and $\mathcal{A}$. Once again, we will prove that *validity*, *consistency*, and *termination* hold perfectly in $\mathcal{E}$ when signatures are modeled ideally in the $\mathcal{F}_{sig}$ -hybrid model, which implies that the properties hold with overwhelming probability over the randomness of $\mathcal{Z}$, $\mathcal{A}$, the activated parties, and the CA, once $\mathcal{F}_{sig}$ is instantiated with EUF-CMA signatures.

Lemma 4.4 (Validity). *Let r be a round number, and let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ be a party who terminates in round r with output a set A_p . Then, $A_p = \{(q, m'_q)\}_{q \in S_p}$, where (1) $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r \subseteq S_p \subseteq \mathcal{P}_{\mathcal{E}}^r$ and (2) for every $q \in S_p$, it holds that $m'_q \in \{0, 1\}^\ell \cup \{\perp\}$, and if $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ then $m'_q = m_q$.*

Proof. For every $u \in S_p \setminus \{p\}$, party p must have received a (p, r') -valid batch of signatures in support of (u, m) for some $m \in \{0, 1\}^\ell$, in some round $r' \leq r$; denote this batch of signatures by B . Thus, by the definition of valid batch of signatures, we have that $\text{Sign}_u(\text{id}_u || m) \in B$. Hence, $u \in \mathcal{P}_{\mathcal{E}}^r$ and thus, by construction of $A = \text{Out}(A_p^r)$, we have that $A_p = \{(q, m'_q)\}_{q \in S_p}$, where $S_p \subseteq \mathcal{P}_{\mathcal{E}}^r$ and $m'_q \in \{0, 1\}^\ell \cup \{\perp\}$ for every $q \in S_p$.

It remains to show that $\{(q, m_q)\}_{q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r} \subseteq A_p$. Notice that $(p, m_p) \in A_p^1$. Moreover, every $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$, with $q \neq p$, sends $\{\{\text{Sign}_q(\text{id}_q || m_q)\}\}$ at the end of round 1 and thus, p receives it in round 2. Hence, p accepts (q, m_q) , since $(q, m_q) \notin A_p^1$ and $\{\text{Sign}_q(\text{id}_q || m_q)\}$ is a $(p, 2)$ -valid batch of signatures for (q, m_q) . Now assume that p accepts any pair (q, m) with $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ and $m \neq m_q$, then p must have received a (p, r') -valid batch of signatures for (q, m) in some round $r' \leq r$, denoted by B' , such that $\text{Sign}_q(\text{id}_q || m) \in B'$. Hence, q has signed $\text{id}_q || m$ at some round prior to r , which is a contradiction, since q is honest until round r . Therefore, $\{(q, m_q)\}_{q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r} \subseteq A_p^r \subseteq A_p$ by construction. $\square$

Our *consistency* proof relies on the fact that let A_p^r and A_q^r be the sets that some parties $p, q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ observe in round r once they decide to terminate (simultaneous termination is guaranteed by Lemma 3.6), then the only way the consistency property may be violated is that there exists some pair (u, m) , such that $(u, m) \in A_p^r \setminus A_q^r$ (otherwise, the Out function will map the sets A_p^r and A_q^r to the same output set). Thus, we will once again prove a relay property, which will now concern the acceptance of party-value pairs.

Lemma 4.5 (Consistency). *Let r be a round number, and let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ be a party which accepts some party-value pair (u, m) in round r . Then, every party $q \in \mathcal{P}_{\mathcal{E}}^1$ that terminates uncorrupted accepts (u, m) by round $r + 1$ before terminating.*

Proof. First, we will prove that the lemma holds for $r = 1$. In round 1, party p only accepts the pair (p, m_p) and sends $\{\{\text{Sign}_p(\text{id}_p || m_p)\}\}$ at the end of round 1. In round 2, every party $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^2$ with $q \neq p$ receives p 's message and accepts $(p || m_p)$, since $p \notin S_q^1$ and $\{\text{Sign}_p(\text{id}_p || m_p)\}$ is a $(q, 2)$ -valid batch of signatures for (p, m_p) .

For round $r = 2$, let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^2$. Note that p accepts (u, m) in round 2 if $u \neq p$ and p receives a $(p, 2)$ -valid batch of signatures for (u, m) of the form $\{\{\text{Sign}_u(\text{id}_u || m)\}\}$ in round 2. In this case, p will send the message $\{\{\text{Sign}_u(\text{id}_u || m)\}, \{\text{Sign}_p(\text{id}_u || m)\}\}$ to all active parties. Now consider a party $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^2$. If $q = u$ then q accepted (u, m) in round 1, and if $q = p$ then q accepted (u, m) in round 2. If $q \notin \{p, u\}$, then either q accepted (u, m) in round 2, or q receives p 's

message in round 3 and accepts (u, m) , since $(u, m) \notin A_q^2$ (since q did not accept (u, m) yet) and $\{\{\text{Sign}_u(\text{id}_u||m)\}, \{\text{Sign}_p(\text{id}_u||m)\}\}$ is a $(q, 3)$ -valid batch of signatures for (u, m) .

Now if $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ accepts (u, m) in round $r \geq 3$ for some $u \in \mathcal{U}$, then p receives a (p, r) -valid batch of signatures for (u, m) in round r , let it be $B = \{\text{Sign}_v(\text{id}_u||m)\}_{v \in V}$, for some $V \subseteq \mathcal{P}$, and observes $(u, m) \notin A_p^{r-1}$ and $|V \cap S_p^{r-1} \setminus \{u\}| \geq r - 2$. So, there exists a set of parties $W \subseteq V \cap S_p^{r-1} \setminus \{u\}$ with $|W| \geq r - 2$, such that p has accepted every party of W by round $r - 1$. By the perfect guarantees of $\mathcal{F}_{\text{sig}}$ we have that $p \notin V$ and thus $p \notin W$ (since $p \in V$ would imply that p 's signature on $\text{id}_u||m$ was forged at some round prior to r). Moreover, let $q \in \mathcal{P}_{\mathcal{E}}^1$ be some party that terminates uncorrupted with $q \neq p$, then using Lemma 3.5 for all parties in W , we have that q accepts all parties of W by round r . Now, consider two cases:

1. If q has accepted (u, m) by round r , then this completes the proof.
2. If q has not accepted (u, m) by round r , we can use the same argument we used for p in order to show that $q \notin W$. So, let $r' \leq r$ be the last round in which q accepts some party of W , then, since q observes $W \cup \{p, q\} \subseteq S_q^{r'}$ in round r' , it also observes $|S_q^{r''}| \geq |S_q^{r'}| \geq r > r'' - 1$ in every round $r'' \in [r', r]$ and thus, participates in round $r + 1$. Since p is honest in round r , it sends a message at the end of round r which is well-formed by construction and contains some batch of signatures for (u, m) , let it be $B' = \{\text{Sign}_v(\text{id}_u||m)\}_{v \in V'}$, with $V \cup \{p\} \subseteq V'$. Therefore, q receives B' in round $r + 1$ and observes $(u, m) \notin A_q^r$ and $|V' \cap S_q^r \setminus \{u\}| \geq |W \cup \{p\}| \geq r - 1$. Thus, q accepts (u, m) in round $r + 1$. $\square$

Finally, note that *simultaneous termination* for the parties of $\mathcal{P}_{\mathcal{E}}^1$ that terminate uncorrupted is implied by Lemmas 3.6 and 3.7.

Lemma 4.6 (Simultaneous Termination). *Let $R_T \in \mathbb{N}$ be the round number when $\mathcal{Z}$ terminates, then all parties in $\mathcal{P}_{\mathcal{E}}^1$ that terminate uncorrupted, terminate simultaneously in at most $|\mathcal{P}_{\mathcal{E}}^{R_T}|$ rounds.*

By Lemmas 4.4, 4.5, and 4.6, we conclude the proof of Theorem 4.3. $\square$

5 Byzantine Broadcast with Unknown Participants

In this section, we will present our UP broadcast protocol. Our protocol will use as a subroutine the UPIC from Section 4.

High-level idea. In accordance with the classic model, UP broadcast can easily be reduced to UPIC, by asking from the parties to run an instance of the latter, where s inputs m_s , while the (rest of) parties input a default value. Once the UPIC execution is completed, then the parties can decide m if (s, m) is included in their output set and $\perp$, otherwise. In this way, the properties of *validity*, *consistency*, and *termination* of UPIC guarantee the corresponding ones of Broadcast. We give a formal description of this protocol in Figure 3.

Theorem 5.1. *Assuming the existence of EUF-CMA signatures, Protocol π_{bc} (Figure 3) is an adaptively secure UP Broadcast protocol (according to Definition 2.2) in the CA-model.*

Moreover, in every execution $\mathcal{E}$ of π_{bc} , all parties in $\mathcal{P}_{\mathcal{E}}^1$ that terminate uncorrupted, terminate simultaneously in at most $|\mathcal{P}_{\mathcal{E}}^{R_T}|$ rounds, where R_T is the last round in which the environment activates new parties.

Protocol π_{bc}

Setup Parameters: The protocol is parameterized by the identity of the sender s , and by the UPIC protocol π_{ic} in the CA model (Figure 2).

UPIC: Every party $p \in \mathcal{U}$ that is activated starts running π_{ic} ; the sender s participates with its input m_s , and all other parties participate with input $\perp$.

Output: Upon receiving the output A_p from π_{ic} , every active party $p \in \mathcal{U}$ outputs m if $(s, m) \in A_p$, for some $m \in \{0, 1\}^\ell$; otherwise, p outputs $\perp$.

Figure 3: UP Broadcast Protocol.

Proof. Let $\mathcal{Z}$ be a dynamic PPT environment and let $\mathcal{A}$ be an adaptive PPT adversary. Let $\mathcal{E}$ be an execution of π_{bc} under $\mathcal{Z}$ and $\mathcal{A}$. Let r be a round number, and let $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ be a party who terminates in round r with output m . Then, by construction $m \in \{0, 1\}^\ell \cup \{\perp\}$, and if $s \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$, by Lemma 4.4, we have that $(s, m_s) \in A_p$ and thus, $m = m_s$. In addition, if $s \notin \mathcal{P}_{\mathcal{E}}^r$, then by Lemma 4.4, we have that $(s, \cdot) \notin A_p$ and therefore, $m = \perp$. Hence, validity of UP Broadcast is preserved. Moreover, by Lemma 4.6, we have that all parties in $\mathcal{P}_{\mathcal{E}}^1$ that terminate uncorrupted, terminate simultaneously in some round r . Therefore, every $q \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^r$ terminate in round r , and by Lemma 4.5, we have that they obtain the same output from π_{ic} . Hence, they output the same value, by construction. $\square$

6 Round-Complexity Lower Bounds

In this section, we provide lower bounds for all three problems that we have studied: UP Broadcast, APSA and UPIC. In particular, we show that any protocol that solves either of these problems, requires at least $|\mathcal{P}_{\mathcal{E}}|$ rounds in a worst case execution $\mathcal{E}$. Since we want to present the strongest possible results, we prove our lower bounds assuming static $\mathcal{Z}$ and $\mathcal{A}$; hence, we can refer to the duration of an execution as the number of rounds required so then all honest parties output a value. In Section 6.1 we present the lower bound for UP Broadcast, and in Section 6.2, for APSA and UPIC.

6.1 A Lower Bound for UP Broadcast

We start by showing that for any *randomized* protocol that solves even the binary version, where the input domain is $\{0, 1\}$, of UP Broadcast in less than $|\mathcal{P}_{\mathcal{E}}|$ rounds in every execution $\mathcal{E}$, there exists a $(\mathcal{Z}, \mathcal{A})$ pair that makes the protocol error with non-negligible probability.

Our argument adopts the approach of Garay *et al.* [GKKO07], who showed that in the known-participants model every randomized broadcast protocol that is secure against $n - 2$ corruptions requires at least $n - 1$ rounds. Particularly, we prove that in the UP setting, the parties may actually need to run for at least one more round, which corresponds to $|\mathcal{P}_{\mathcal{E}}|$ rounds overall.

In fact, while in the classic model the honest parties can make sure they have consistent views with each other about the behaviour of the sender in $n - 1$ rounds, in the UP model, when the sender might not be active there may exist some honest party which is aware of the participation of n parties (including itself) by round $n - 1$, but is not certain whether these parties have received an input from the sender or not. More precisely, the party may not be able to distinguish between the scenario where the sender is not active and the one where the sender's message has been received

by all other parties except for itself. Hence, this party has to run for at least one more round to resolve the situation.

We begin by presenting in Lemma 6.1 the core idea in the simpler case where the adversary corrupts all but two honest parties. Next, in Theorem 6.5 we extend to the more general case.

Lemma 6.1. *There exist a static environment $\mathcal{Z}$ and a static adversary $\mathcal{A}$ such that the following holds for every randomized UP Broadcast protocol π : In a random execution $\mathcal{E}$ with $\mathcal{Z}$ and $\mathcal{A}$, in which $\mathcal{Z}$ activates party-set $\mathcal{P}_{\mathcal{E}}$ and $\mathcal{A}$ corrupts all but two parties, it holds that at least one honest party in $\mathcal{E}$ runs for at least $|\mathcal{P}_{\mathcal{E}}|$ rounds, except with negligible probability (where the probability is over the coins of the environment, the adversary, and the honest parties).*

Proof. Let π be a UP Broadcast protocol and, without loss of generality, assume that the sender is p_1 . Given the security parameter κ , consider the set of parties $\mathcal{P} = \{p_1, \dots, p_{\kappa}\}$, and define the following two static environments:

- $\mathcal{Z}_1$ activates all parties in $\mathcal{P}$.
- $\mathcal{Z}_2$ activates the parties in $\mathcal{P} \setminus \{p_1\}$.

Next, define κ adversaries, $\mathcal{A}_1, \dots, \mathcal{A}_{\kappa}$, as follows:

- For $i \in [\kappa - 1]$, $\mathcal{A}_i$ corrupts all parties in $\mathcal{P} \setminus \{p_i, p_{i+1}\}$ (if they are active).
- $\mathcal{A}_{\kappa}$ corrupts the parties in $\mathcal{P} \setminus \{p_{\kappa-1}, p_{\kappa}\}$ (if they are active).

For every $i \in [\kappa]$, the adversary $\mathcal{A}_i$ instructs every corrupt p_j to follow π ; however, $\mathcal{A}_i$ cuts all communications channels between p_j and parties other than p_{j-1} and p_{j+1} (if they are active).⁷

That is, $\mathcal{A}_i$ drops every message sent by p_j to parties other than p_{j-1} and p_{j+1} , and every message sent to p_j from parties other than p_{j-1} and p_{j+1} .

Given the above adversaries, we define the following scenarios (illustrated in Figure 4):

- For $i \in [\kappa - 1]$, scenario S_i corresponds to executing π with environment $\mathcal{Z}_1$ and adversary $\mathcal{A}_i$. Note that for every execution $\mathcal{E}$ of π in scenario S_i , the number of active parties is $|\mathcal{P}_{\mathcal{E}}| = \kappa$, out of which $|\mathcal{H}_{\mathcal{E}}| = 2$ are honest.
- Scenario S_{κ} corresponds to executing π with environment $\mathcal{Z}_2$ and adversary $\mathcal{A}_{\kappa}$. For every execution $\mathcal{E}$ of π in scenario S_{κ} , the number of active parties is $|\mathcal{P}_{\mathcal{E}}| = \kappa - 1$, out of which $|\mathcal{H}_{\mathcal{E}}| = 2$ parties are honest.

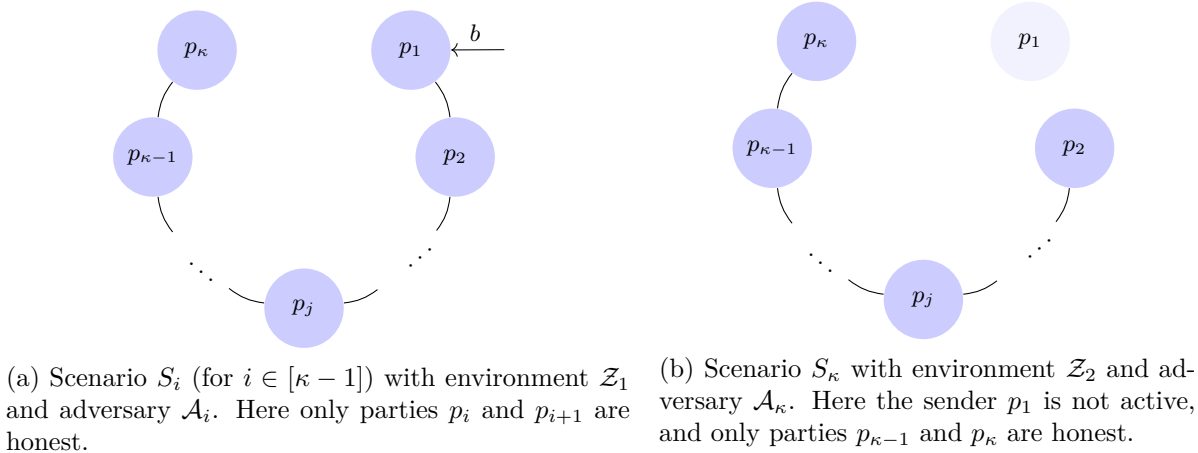


Figure 4: Configurations corresponding to scenarios $S_1, \dots, S_{\kappa}$. Every p_j communicates only with parties p_{j-1} and p_{j+1} (if active).

⁷By abuse of notation, we consider parties p_0 and $p_{\kappa+1}$ (which are not defined in $\mathcal{P}$) as not active.

Let $b \leftarrow \{0, 1\}$ be a uniformly distributed bit. Towards contradiction, assume that in every execution $\mathcal{E}$ of π in scenarios $S_1, \dots, S_\kappa$, where the sender p_1 holds the input bit b , all honest parties terminate in $|\mathcal{P}_\mathcal{E}| - 1$ rounds.

Claim 6.2. *In a random execution $\mathcal{E}$ of scenario $S_{\kappa-1}$, party p_κ outputs b with overwhelming probability.*

Proof. First, note that in scenario S_1 , parties p_1 and p_2 are the only honest parties. Hence, by validity, in a random execution of S_1 both parties output b except for negligible probability. Second, for every $i \in \{2, \dots, \kappa - 1\}$ the view of party p_i is identically distributed between scenarios S_i and S_{i-1} , and therefore the output distribution of party p_i is identically distributed in scenarios S_i and S_{i-1} . Third, for every $i \in \{2, \dots, \kappa - 1\}$, by consistency, in a random execution of Scenario S_i honest parties p_i and p_{i+1} output the same value except for negligible probability. By a standard hybrid argument, we conclude that in a random execution of scenario $S_{\kappa-1}$, party p_κ outputs b except for negligible probability. $\square$

Claim 6.3. *The output of p_κ is identically distributed in scenarios $S_{\kappa-1}$ and S_κ .*

Proof. Let $\mathcal{E}_\kappa$ be a random execution of Scenario S_κ ; it holds that $|\mathcal{P}_{\mathcal{E}_\kappa}| = \kappa - 1$, out of which $|\mathcal{H}_{\mathcal{E}_\kappa}| = 2$ parties are honest. By assumption, honest party p_κ terminates in at most $|\mathcal{P}_{\mathcal{E}_\kappa}| - 1 = \kappa - 2$ rounds. Now, let $\mathcal{E}_{\kappa-1}$ be a random execution of Scenario $S_{\kappa-1}$. Since p_κ is at distance $\kappa - 1$ hops from p_1 in $S_{\kappa-1}$, it holds that the view of p_κ is identically distributed in Scenarios S_κ and $S_{\kappa-1}$. Hence, in scenario $S_{\kappa-1}$ party p_κ also terminates in at most $\kappa - 2$ rounds, and its output is identically distributed in scenarios $S_{\kappa-1}$ and S_κ . $\square$

Claim 6.4. *In a random execution $\mathcal{E}$ of scenario S_κ , party p_κ outputs b with probability $1/2$.*

Proof. In Scenario S_κ the sender p_1 is not activated. Hence, the view of p_κ is independent of b . $\square$

Combined together, Claims 6.2, 6.3, and 6.4 yield a contradiction. This concludes the proof of Lemma 6.1. $\square$

In the spirit of [GKKO07], we generalize the lower bound to show that any UP Broadcast protocol requires at least $\lfloor 2|\mathcal{P}_\mathcal{E}|/|\mathcal{H}_\mathcal{E}| \rfloor$ rounds when $|\mathcal{H}_\mathcal{E}| \geq 2$.

Theorem 6.5. *There exist a static environment $\mathcal{Z}$ and a static adversary $\mathcal{A}$ such that the following holds for every randomized UP Broadcast protocol π : In a random execution $\mathcal{E}$ with $\mathcal{Z}$ and $\mathcal{A}$, in which $\mathcal{Z}$ activates party-set $\mathcal{P}_\mathcal{E}$ and $\mathcal{A}$ corrupts all but a subset of parties $\mathcal{H}_\mathcal{E}$, the honest parties in $\mathcal{H}_\mathcal{E}$ must run for at least $\lfloor 2|\mathcal{P}_\mathcal{E}|/|\mathcal{H}_\mathcal{E}| \rfloor$ rounds, except for negligible probability.*

Proof. Let π be a UP Broadcast protocol and assume, without loss of generality, that the sender is p_1 . Let f_1 and f_2 be two polynomially bounded functions satisfying $f_2(\kappa) < f_1(\kappa)$ for every $\kappa \in \mathbb{N}$ (for simplicity of notation, assume that $f_2(\kappa)$ is even and divides $2f_1(\kappa)$). Assume that for every static environment $\mathcal{Z}$ that activates $n = f_1(\kappa)$ parties and for every static adversary $\mathcal{A}$ that corrupts all but $h = f_2(\kappa)$ parties, in every execution $\mathcal{E}$ the protocol π terminates in less than $2n/h$ rounds.

We now derive another UP Broadcast protocol π_1 from π . Let $\mathcal{Z}_1$ be a static environment that activates the set of parties $\mathcal{Q} = \{q_1, \dots, q_{n_1}\}$. Each party q_i in π_1 computes $n = f_1(\kappa)$ and $h = f_2(\kappa)$ and emulates in its heed $h/2$ virtual parties in π , denoted for simplicity $p_{(i-1)+1}, \dots, p_{(i-1)+h/2}$. The sender q_1 sets the input bit of the virtual sender p_1 according to its own input. Whenever a party q_i emulates a virtual party p_j sending a message m to virtual party p_k in round r , then party

q_i sends the message (j, k, r, m) to the party emulating p_k (and emulates it in its head if q_i emulates p_k). Finally, once all virtual parties emulated by q_i terminate, q_i outputs the value of the $p_{(i-1)+1}$ and terminates.

Clearly, if π is secure against static adversaries that corrupt up to $h = f_2(\kappa)$ parties, then π_1 is secure against static adversaries that corrupt up to 2 parties. In the virtual execution of π there are $n = n_1 \cdot h/2$ parties, and if the adversary corrupts all but two, there are at least $2 \cdot h/2 = h$ uncorrupted parties in π . By assumption, the number of round in which π terminates is less than

$$\frac{2n}{h} = \frac{2n_1 \cdot h/2}{h} = n_1.$$

Hence, we obtain that in such executions π_1 has n_1 active parties, out of which all but two are corrupted, and terminates in less than n_1 rounds. This contradicts Lemma 6.1. $\square$

6.2 Lower Bounds for APSA and UPIC

We will now prove that the $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ lower bound also holds for both APSA and UPIC protocols. The round-preserving reductions that we will present right after indicate a hierarchy among the problems that we defined in the UP model, which is shown in Figure 5, where the arrows are pointing to easier problems from harder ones.

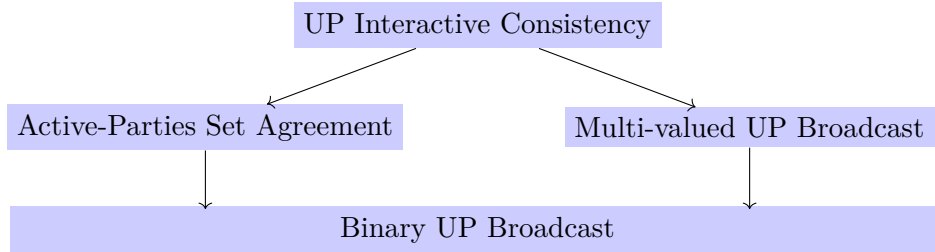


Figure 5: Hierarchy of problems in the UP model.

Although, the reduction from (multi-valued) UP Broadcast to UP Interactive Consistency is rather obvious, it turns out that the binary version of UP Broadcast can also be immediately reduced to APSA.

Corollary 6.6. *There does not exist any randomized APSA protocol that terminates (with overwhelming probability) in fewer than $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ rounds in every execution $\mathcal{E}$.*

Proof. Assume for the contrary that there exists a randomized APSA protocol π that terminates with overwhelming probability in fewer than $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ rounds, then we can construct a randomized UP Broadcast protocol $\pi_{\text{bc-apsa}}$ as in Figure 6.

Notice that if s is honest and has input value $b_s = 1$, then it participates in π and by the validity and consistency properties of APSA, with overwhelming probability every honest party includes s in its output set. Hence, every honest party outputs 1. If s is honest and has input value $b_s = 0$, then it behaves as inactive, and thus, by validity of APSA, with overwhelming probability s is not included in the output of any honest party. Therefore, every honest party outputs 0. Finally, consistency and termination in at most $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ rounds of UP Broadcast are satisfied with same probability as the corresponding properties of APSA. Hence, $\pi_{\text{bc-apsa}}$ satisfies all properties of UP Broadcast protocol and also terminates in fewer than $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ rounds in any execution $\mathcal{E}$, with overwhelming probability overall, which contradicts Theorem 6.5. $\square$

Protocol $\pi_{\text{bc-apsa}}$	
Setup Parameters:	The protocol is parameterized by the identity of the sender s , and by an APSA protocol π .
Active-Parties Set Agreement:	Every active party $p \in \mathcal{U} \setminus \{s\}$ starts running π ; if the sender s is active and has input $b_s = 1$, then s participates honestly in π , whereas if $b_s = 0$, the sender s remains silent throughout the execution.
Output:	Upon receiving the output S_p from π , every active party $p \in \mathcal{U}$ outputs 1 if $s \in S_p$, and otherwise outputs 0.

Figure 6: Reduction from Binary UP Broadcast to APSA.

The reduction from UP Broadcast to UPIC is already shown in the security proof of our UP Broadcast protocol in Section 5.

Corollary 6.7. *There does not exist any randomized UPIC protocol that terminates (with overwhelming probability) in fewer than $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ rounds in every execution $\mathcal{E}$.*

Proof. Assume for the contrary that there exists a randomized UPIC protocol π that terminates with overwhelming probability in fewer than $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ rounds then we can construct a UP Broadcast protocol as in Figure 3, which also terminates in fewer than $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ rounds with overwhelming probability in every execution $\mathcal{E}$, which contradicts Theorem 6.5. $\square$

Tightness of our Protocols in the UP Model. Once we have proved that the problems of APSA, UP Broadcast, and UPIC all require at least $\lfloor 2|\mathcal{P}_{\mathcal{E}}|/|\mathcal{H}_{\mathcal{E}}| \rfloor$ rounds in the worst-case, we can claim that the protocols that we presented in Sections 3 and 4 all achieve optimal worst-case round complexity. Recall, that all our protocols in the UP model require at most $|\mathcal{P}_{\mathcal{E}}^{R_T}|$ rounds in every execution $\mathcal{E}$, which equals the number of rounds indicated by the lower bounds above for the (worst-)case that $|\mathcal{H}_{\mathcal{E}}^{R_T}| = 2$.

7 Late-Joining Parties

So far in this work, the protocols that we have constructed and analyzed in the UP model satisfy properties regarding honest parties activated only from the beginning of the execution, and only give guarantees to such parties.. A natural question is whether we can provide some guarantees for parties activated *after* the beginning of the execution and which remain honest until they terminate. Our main goal is to preserve the properties discussed in the previous sections with respect to the parties in $\mathcal{P}_{\mathcal{E}}^1$, while providing some guarantees for the parties in $\mathcal{P}_{\mathcal{E}}^r$ for $r > 1$. We will start by investigating in Section 7.1, security for late-joining *receivers*, and in Section 7.2, security for late-joining *sender*.

7.1 Security for Late-Joining Receivers

We begin by presenting an impossibility result for a natural extension of Definition 2.2 in Section 7.1.1, followed by a protocol achieving security with “selective abort” in Section 7.1.2.

7.1.1 Impossibility of UP Broadcast with Late-Joining Receivers

We begin by analyzing a strong variant of UP Broadcast with late-joining receivers, in which all parties that are honest in round r and terminate in round r , must output the same value as parties that were activate in the first round and remain honest until their termination. Ideally, we would like the late receivers to be able to achieve this regardless of how lately they join the execution as long as none of the parties in $\mathcal{P}_\mathcal{E}^1$ has terminated uncorrupted by the time they join.

Definition 7.1 (UP Broadcast with Late-Joining Receivers). *Let $\kappa \in \mathbb{N}$ be a security parameter, let $\mathcal{U}$ be a (possibly exponential) set of parties, and let $s \in \mathcal{U}$. A protocol π is an **adaptively secure, late-joining receivers UP Broadcast protocol with sender s** , if for every dynamic PPT environment $\mathcal{Z}$, every adaptive PPT adversary $\mathcal{A}$, and every input message $m_s \in \{0,1\}^\ell$ with $\ell = O(\kappa)$, with overwhelming probability, an execution $\mathcal{E}$ of π with sender input m_s in the presence of $\mathcal{Z}$ and $\mathcal{A}$ satisfies the following: let r^* be the last round such that no party in $\mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r^*}$ has terminated by round r^* , then:*

- **Validity:** *Let r be a round number, and let $p \in \mathcal{P}_\mathcal{E}^{r^*} \cap \mathcal{H}_\mathcal{E}^r$ be a party who terminates in round r with output m . Then $m \in \{0,1\}^\ell \cup \{\perp\}$, where $m = m_s$ if $s \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^r$, and $m = \perp$ if $s \notin \mathcal{P}_\mathcal{E}^1$.*
- **Consistency:** *Let r_p and r_q be round numbers, and let $p \in \mathcal{P}_\mathcal{E}^{r^*} \cap \mathcal{H}_\mathcal{E}^{r_p}$ and $q \in \mathcal{P}_\mathcal{E}^{r^*} \cap \mathcal{H}_\mathcal{E}^{r_q}$ be two parties that terminate in rounds r_p and r_q with outputs m_p and m_q , respectively. Then $m_p = m_q$.*
- **Termination:** *All parties in $\mathcal{P}_\mathcal{E}^{r^*}$ either terminate in polynomially (in κ) many rounds, or they get corrupted before they terminate.*

Notice that $r^* \geq 1$ and thus, the requirements for the parties in $\mathcal{P}_\mathcal{E}^1$ that terminate uncorrupted are the same as in our UP Broadcast definition (Definition 2.2) and that the same requirements also apply to parties that join before any of the parties in $\mathcal{P}_\mathcal{E}^1$ terminate uncorrupted.

Inspired by the resiliency bound presented by Pass and Shi in [PS17], we begin by showing that without applying any further restrictions to the joining time of the late receivers, no deterministic protocol can meet the above definition. More precisely, we show that if there exists a late-joining receiver that joins in round r^* and terminates uncorrupted, then it is impossible that π satisfies all the required properties of Definition 7.1.

Lemma 7.2. *There does not exist any deterministic late-joining receivers broadcast protocol π in the UP setting that satisfies the properties of Definition 7.1.*

Proof. Assume to the contrary that there exists a deterministic late-joining receivers UP Broadcast protocol π . For a fixed integer $n \geq 2$, let $s = p_1$, let $G_1 = \{p_2, \dots, p_{n+1}\}$, and let $G_2 = \{p_{n+2}, \dots, p_{2n+1}\}$. We consider the following executions of π for $m, m' \in \{0,1\}^\ell$:

- $\mathcal{E}_{1,m}$: $\mathcal{Z}_1$ activates $G_1 \cup \{s\}$ at the beginning of the execution and the sender's input is m . $\mathcal{A}_1$ corrupts no party. Let $r_{1,m}$ denote the first round when a party of G_1 terminates.
- $\mathcal{E}_{2,m'}$: $\mathcal{Z}_2$ activates $G_2 \cup \{s\}$ at the beginning of the execution and the sender's input is m' . $\mathcal{A}_2$ corrupts no party. Let $r_{2,m'}$ denote the first round when a party of G_2 terminates.

Since all executions must be completed in $O(\text{poly}(\kappa))$ rounds and there are 2^ℓ choices of choosing (m, m') , there exist $m \neq m'$ such that $r_{1,m} = r_{2,m'}$; denote this round by r . We fix such a pair (m, m') and denote $\mathcal{E}_1 = \mathcal{E}_{1,m}$ and $\mathcal{E}_2 = \mathcal{E}_{2,m'}$ for convenience. In $\mathcal{E}_1$ and $\mathcal{E}_2$, by validity of π , all parties in G_1 output m and all parties in G_2 output m' . Let $p'_1 \in G_1$ and $p'_2 \in G_2$ be parties that terminate in round r in $\mathcal{E}_1$ and $\mathcal{E}_2$, respectively. Fix an arbitrary $q \in \mathcal{U} \setminus (G_1 \cup G_2 \cup \{s\})$. Consider the following executions $\mathcal{E}_3$ and $\mathcal{E}_4$, defined with $(\mathcal{Z}_3, \mathcal{A}_3)$ and $(\mathcal{Z}_4, \mathcal{A}_4)$, respectively:

- $\mathcal{E}_3$: $\mathcal{Z}_3$ activates $G_1 \cup G_2 \cup \{s, q\}$ at the beginning of the execution but $\mathcal{A}_3$ immediately takes control over $G_2 \cup \{s, q\}$. Particularly, $\mathcal{A}_3$ forces the sender to behave honestly towards the parties of G_1 as if it is given m as an input value. $\mathcal{A}_3$ also simulates the execution where the parties of $G_2 \cup \{s\}$ communicate honestly with each other and s has been given m' as an input value, dropping any messages sent by G_1 to G_2 and vice versa. Finally, it forces q to remain silent until round $r - 2$ and behave like a late-joining receiver that joins in round $r - 1$, except that it does not send any messages to p'_1 .
- $\mathcal{E}_4$: $\mathcal{Z}_4$ activates $G_1 \cup G_2 \cup \{s, q\}$ at the beginning of the execution but $\mathcal{A}_4$ immediately takes control over $G_1 \cup \{s, q\}$. Particularly, $\mathcal{A}_4$ forces the sender to behave honestly towards the parties of G_2 as if it is given m' as an input value. $\mathcal{A}_4$ also simulates the execution where the parties of $G_1 \cup \{s\}$ communicate honestly with each other and s has been given m as an input value, dropping any messages sent by G_2 to G_1 and vice versa. Finally, it forces q to remain silent until round $r - 2$, and behave like a late-joining receiver that joins in round $r - 1$, except that it does not send any messages to p'_2 .

Hence, p'_1 cannot distinguish between $\mathcal{E}_1$ and $\mathcal{E}_3$ by round r and thus, cannot terminate in round r with output m in $\mathcal{E}_3$. Accordingly, p'_2 cannot distinguish between $\mathcal{E}_2$ and $\mathcal{E}_4$ by round r and thus, cannot terminate in round r with output m' in $\mathcal{E}_4$. By consistency of π , all parties in G_1 output m in $\mathcal{E}_3$ and all parties in G_2 output m' in $\mathcal{E}_4$. Now, consider the following executions $\mathcal{E}_5$ and $\mathcal{E}_6$, defined with $(\mathcal{Z}_5, \mathcal{A}_5)$ and $(\mathcal{Z}_6, \mathcal{A}_6)$, respectively:

- $\mathcal{E}_5$: $\mathcal{Z}_5$ activates $G_1 \cup G_2 \cup \{s\}$ at the beginning of the execution but $\mathcal{A}_5$ immediately takes control over $G_2 \cup \{s\}$. Particularly, $\mathcal{A}_5$ forces the sender to behave honestly towards the parties of G_1 as if it is given m as an input value. $\mathcal{A}_5$ also simulates the execution where the parties of $G_2 \cup \{s\}$ communicate honestly with each other and s has been given m' as an input value, dropping any messages sent by G_1 to G_2 and vice versa. Finally, $\mathcal{Z}_5$ activates q in round $r - 1$ and $\mathcal{A}_5$ corrupts p'_1 in round r and forces it to remain silent in every round $\geq r$.
- $\mathcal{E}_6$: $\mathcal{Z}_6$ activates $G_1 \cup G_2 \cup \{s\}$ at the beginning of the execution but $\mathcal{A}_6$ immediately takes control over $G_1 \cup \{s\}$. Particularly, $\mathcal{A}_6$ forces the sender to behave honestly towards the parties of G_2 as if it is given m' as an input value. $\mathcal{A}_6$ also simulates the execution where the parties of $G_1 \cup \{s\}$ communicate honestly with each other and s has been given m as an input value, dropping any messages sent by G_2 to G_1 and vice versa. Finally, $\mathcal{Z}_6$ activates q in round $r - 1$ and $\mathcal{A}_6$ corrupts p'_2 in round r and forces it to remain silent in every round $\geq r$.

Notice that the parties in $G_1 \setminus \{p'_1\}$ have the same view in $\mathcal{E}_3$ and $\mathcal{E}_5$ and thus, they output m in $\mathcal{E}_5$ as well. Similarly, the parties in $G_2 \setminus \{p'_2\}$ output m' in $\mathcal{E}_6$. So, by consistency of π , party q outputs m in $\mathcal{E}_5$ and since q cannot distinguish between $\mathcal{E}_5$ and $\mathcal{E}_6$, it also outputs m in $\mathcal{E}_6$. However, this violates consistency in $\mathcal{E}_6$, a contradiction. $\square$

7.1.2 UP Broadcast with Selective abort for Late-Joining Receivers

We proceed to present a weaker notion of UP Broadcast, which enables late-joining receivers to abort in case they identify that an attack is taking place. Non-triviality is captured by requiring that if all parties are honest they must output the sender's input as long as they join before all parties that were active in the first round terminate. Borrowing terminology from the MPC realm [GL05, IKP10], we call this variant UP Broadcast with *selective abort*, and note that requiring security for late-joining receivers *only* when all active parties are honest, also resembles the notion of *weak broadcast* [Lam83].

Definition 7.3 (UP Broadcast with Selective Abort for Late-Joining Receivers). *Let $\kappa \in \mathbb{N}$ be a security parameter, let $\mathcal{U}$ be a (possibly exponential) set of parties, and let $s \in \mathcal{U}$. A protocol π is an adaptively secure, late-joining receivers UP Broadcast protocol with selective abort with sender s , if for every dynamic IPT environment $\mathcal{Z}$, every adaptive IPT adversary $\mathcal{A}$, and every input message $m_s \in \{0,1\}^\ell$ with $\ell = O(\kappa)$, with overwhelming probability, an execution $\mathcal{E}$ of π with sender input m_s in the presence of $\mathcal{Z}$ and $\mathcal{A}$ satisfies the following: let r^* be the last round such that no party in $\mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r^*}$ has terminated by round r^* , then:*

- **Validity:** *Let r be a round number, and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^r$ be a party who terminates in round r with output m . Then $m \in \{0,1\}^\ell \cup \{\perp\}$, where $m = m_s$ if $s \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^r$, and $m = \perp$ if $s \notin \mathcal{P}_\mathcal{E}^1$.*
- **Consistency:** *Let r_p and r_q be round numbers, and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_p}$ and $q \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_q}$ be two parties that terminate in rounds r_p and r_q with outputs m_p and m_q , respectively. Then $m_p = m_q$.*
- **Late-joining consistency:** *Let r_p and r_q be round numbers and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_p}$ and $q \in \mathcal{P}_\mathcal{E}^{r_q} \cap \mathcal{H}_\mathcal{E}^{r_q}$ be two parties that terminate in rounds r_p and r_q with outputs m_p and m_q , respectively. Then, $m_q \in \{m_p, \perp\}$ and furthermore, if $\mathcal{P}_\mathcal{E}^{r_q} = \mathcal{H}_\mathcal{E}^{r_q}$, then $m_q = m_p$.*
- **Termination:** *All parties in $\mathcal{P}_\mathcal{E}^{r^*}$ either terminate in polynomially (in κ) many rounds or they get corrupted before they terminate.*

Notice that the properties of *validity* and *consistency* are the same as in our UP Broadcast definition. The *late-joining consistency* guarantees that if a late receiver joins before any of the parties in $\mathcal{P}_\mathcal{E}^1$ (that remain honest) has terminated, then it will either output the same value with the parties in $\mathcal{P}_\mathcal{E}^1$ that terminate uncorrupted, or **abort**.

UP Broadcast with selective abort. Our starting point is the UPIC protocol from Section 4. Note that in this protocol, if all active parties remain honest throughout the execution, then no communication is required after round 3. Indeed, all parties send their initial messages in round 1 and forward the received signatures along with their own ones in round 2. Hence, when considering late-joining receivers, the parties activated from the beginning can simply send in every round r their A_p^r set (defined in Figure 2), so then if a late-joining party receives $|A|$ identical copies of a set A , it can temporarily assume that the commonly agreed set is A . If no other message, which would reveal the presence of at least one active corruption, is received by the late-joining party in $|A|$ rounds from the beginning of the execution, then it can safely output m if (s, m) is the only pair in A with respect to s , and output $\perp$ otherwise. In any other case, it is allowed to abort the execution.

Notice that with this approach, late-joining parties remain silent and thus, they do not affect the UPIC instance. So, if no corruptions occur then all parties terminate simultaneously in $|A| = |\mathcal{P}_\mathcal{E}^1|$ rounds. The only exceptional case is $|\mathcal{P}_\mathcal{E}^1| = 1$. Then, the adversary can confuse the only party in $\mathcal{P}_\mathcal{E}^1$, let it be p , and the late-joining parties by introducing to p a new value right before it terminates while not sending anything to the late-joiner at the same time. In this case, p may disagree with the late-joiners in the rounds to follow. We can easily overcome this issue by reasonably assuming that there are at least two parties in $\mathcal{P}_\mathcal{E}^1$; otherwise, consistency is meaningless. Our selective abort protocol is described in Figure 7.

Note that if the protocol terminates in round $|A| + 1$, it takes $|A|$ rounds of communication. We will now show that the above protocol achieves UP Broadcast with selective abort for late-joining receivers.

Protocol $\pi_{\text{bc-sa}}$

Setup Parameters: The protocol is parameterized by the identity of the sender s , and by the UPIC protocol π_{ic} (Figure 2).

UPIC: In round 1, every active party $p \in \mathcal{U} \setminus \{s\}$ starts running a π_{ic} instance with input $\perp$ and s participates with input m_s . In every round $r \geq 1$, every active party $p \in \mathcal{U}$ attaches A_p^r to the message it sends (if any) at the end of r .

Selective Abort: Let $p \in \mathcal{U}$ be a party activated in round $r > 1$. If p receives a message M that is not a set of party-value pairs or two messages A and A' with $A \neq A'$ in round $r + 1$, then it outputs **abort**. Otherwise, let A be the only set that p received in round $r + 1$, then whenever p receives a message other than A by round $|A| + 1$, it outputs **abort**. Otherwise, p terminates in round $|A| + 1$ with output m , if $(s, m) \in A$ for an $m \in \{0, 1\}^\ell$ and $\perp$, otherwise.

Figure 7: UP Broadcast protocol with Selective Abort for Late-Joining Receivers.

Theorem 7.4. *Assuming the existence of EUF-CMA signatures, Protocol $\pi_{\text{bc-sa}}(\pi_{\text{ic}})$ (Figure 7) is an UP broadcast protocol with selective abort for late-joining receivers (according to Definition 7.3) in the CA-model. Moreover, in every execution $\mathcal{E}$ of $\pi_{\text{bc-sa}}(\pi_{\text{ic}})$, let r^* be the last round such that no party in $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{r^*}$ has terminated by round r^* , then all parties in $\mathcal{P}_{\mathcal{E}}^{r^*}$ that terminate uncorrupted, terminate in at most $|\mathcal{P}_{\mathcal{E}}^{R_T}|$ rounds from the beginning of the execution.*

Proof. First, note that the underlying π_{ic} instance is not affected by the messages regarding the treatment of late-joining parties and thus, the properties of validity and consistency are maintained. Now, if in an execution $\mathcal{E}$ it holds that $\mathcal{H}_{\mathcal{E}}^{|\mathcal{P}_{\mathcal{E}}^1|+1} = \mathcal{P}_{\mathcal{E}}^{|\mathcal{P}_{\mathcal{E}}^1|+1}$, then by the properties of π_{ic} we have that all parties in $\mathcal{P}_{\mathcal{E}}^1$ terminate simultaneously in round $|\mathcal{P}_{\mathcal{E}}^1| + 1$. Moreover, every party $p \in \mathcal{P}_{\mathcal{E}}^1$ observes $A_p^r = A$, in every round $r \geq 2$ and thus, A is the only message that every party $q \in \mathcal{H}_{\mathcal{E}}^{|\mathcal{P}_{\mathcal{E}}^1|+1}$ receives until round $|\mathcal{P}_{\mathcal{E}}^1| + 1$. Therefore, p and q output the same message.

Moreover, assume that for rounds r_p and r_q , there exist parties $p \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{r_p}$ and $q \in \mathcal{P}_{\mathcal{E}}^{r_q} \cap \mathcal{H}_{\mathcal{E}}^{r_q}$ which terminate in r_p and r_q with outputs m_p and m_q , respectively, such that $m_p \neq m_q$. Then q received a set A such that $(s, m_q) \in A$ and no sets other than A . Thus, p sent A in a round $r < r_p$. Since $p \in \mathcal{H}_{\mathcal{E}}^{r_p}$, p must have accepted a pair (s, m') in a round $r' > r$. If $r' \geq |A| + 1$, then p observed $|A| - 1$ signatures on the newly accepted pair from parties which were included in the pairs of A . So, at least one of these signatures must have been generated by another party $w \in \mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{r'} \setminus \{p\}$ and hence, q also receives the same batch in round $|A| + 1$ before terminating. If $r' \leq |A|$, then p sends a batch for its newly accepted pair and q receives it by round $r' + 1 \leq |A|$ before terminating. Hence, in any case q observes a message other than A before terminating and thus, outputs **abort**, a contradiction. Hence, late-joining consistency of $\pi_{\text{bc-sa}}$ is also satisfied.

Finally, termination in at most $|\mathcal{P}_{\mathcal{E}}^{R_T}|$ rounds is satisfied by the properties of the π_{ic} protocol. $\square$

7.2 Security with a Late-Joining Sender

Similarly with security for late-joining receivers, we will begin by showing in Section 7.2.1 that achieving security for a late-joining sender is impossible in the general case. In Section 7.2.2 we describe a simple protocol when it is guaranteed that the sender has enough time to distribute its message after joining. In Section 7.2.3 we describe a relaxation of the UP model that enables offline communication before the sender obtains its input message, and in Section 7.2.4 we present

an impossibility of UP Broadcast with a late-joining sender that has less time, even in the weaker model.

7.2.1 Impossibility of UP Broadcast with a Late-Joining Sender

In principal, a late-joining sender UP Broadcast protocol should preserve the properties of a UP Broadcast protocol (as defined in Section 2.2), regardless of how lately the sender joins the execution (as long as by the time it joins, no parties have terminated uncorrupted). Hence, we provide the definition of UP Broadcast with security for late-joining sender as follows:

Definition 7.5 (UP Broadcast with a Late-Joining Sender). *Let $\kappa \in \mathbb{N}$ be a security parameter, let $\mathcal{U}$ be a (possibly exponential) set of parties, and let $s \in \mathcal{U}$. A protocol π is an **adaptively secure, late-joining sender UP Broadcast protocol with sender s** , if for every dynamic IPT environment $\mathcal{Z}$, every adaptive IPT adversary $\mathcal{A}$, and every input message $m_s \in \{0, 1\}^\ell$ with $\ell = O(\kappa)$, with overwhelming probability, an execution $\mathcal{E}$ of π with sender input m_s in the presence of $\mathcal{Z}$ and $\mathcal{A}$ satisfies the following: let r^* be the last round such that no party in $\mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r^*}$ has terminated by round r^* , then:*

- **Validity:** *Let r be a round number, and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^r$ be a party who terminates in round r with output m . Then $m \in \{0, 1\}^\ell \cup \{\perp\}$, where $m = m_s$ if $s \in \mathcal{P}_\mathcal{E}^{r^*} \cap \mathcal{H}_\mathcal{E}^r$, and $m = \perp$ if $s \notin \mathcal{P}_\mathcal{E}^{r-1}$.*
- **Consistency:** *Let r_p and r_q be round numbers, and let $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_p}$ and $q \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{r_q}$ be two parties that terminate in rounds r_p and r_q with outputs m_p and m_q , respectively. Then $m_p = m_q$.*
- **Termination:** *All active parties either terminate in polynomially (in κ) many rounds or they get corrupted before they terminate.*

We will now prove that there exists no deterministic protocol that meets the above definition.

Lemma 7.6. *There does not exist any deterministic late-joining sender UP Broadcast protocol π in the UP setting that satisfies the properties of Definition 7.5.*

Proof. Assume toward contradiction that there exists a deterministic late-joining sender UP Broadcast protocol π . Let $s = p_1$ with input m_s and for any value of $n \geq 2$, let $G = \{p_2, \dots, p_{n+1}\}$. Consider the following executions $\mathcal{E}_1$, $\mathcal{E}_2$, and $\mathcal{E}_3$ of π , working with $(\mathcal{Z}_1, \mathcal{A}_1)$, $(\mathcal{Z}_2, \mathcal{A}_2)$, and $(\mathcal{Z}_3, \mathcal{A}_3)$, respectively.

- $\mathcal{E}_1$: $\mathcal{Z}_1$ activates G at the beginning of the execution and keeps s inactive. $\mathcal{A}_1$ corrupts no parties and r is the first round such that there exists a party $p \in G$ that terminates in round r .
- $\mathcal{E}_2$: $\mathcal{Z}_2$ activates G at the beginning of the execution and activates s in round $r - 1$. $\mathcal{A}_2$ only corrupts p in round r and forces it to remain silent ever after.
- $\mathcal{E}_3$: $\mathcal{Z}_3$ activates G at the beginning of the execution and activates s in round $r - 1$. $\mathcal{A}_3$ only corrupts s in round $r - 1$ and forces it to behave honestly towards the parties in $G \setminus \{p\}$, but not send anything to p .

In $\mathcal{E}_1$, due to validity of π , party p terminates with output $\perp$ in round r . Notice that p cannot distinguish between $\mathcal{E}_1$ and $\mathcal{E}_3$ up to round r and thus, terminates in round r of $\mathcal{E}_3$ with output $\perp$ as well. Moreover, the parties in $G \setminus \{p\}$ cannot distinguish between $\mathcal{E}_1$, $\mathcal{E}_2$, and $\mathcal{E}_3$ by round $r - 1$ and thus, s is activated in $\mathcal{E}_2$ and $\mathcal{E}_3$ before any party terminates uncorrupted in either of the two executions. Hence, using the consistency property of π , we have that the parties in $G \setminus \{p\}$ output $\perp$ in $\mathcal{E}_3$, while due to validity, they output m_s in $\mathcal{E}_2$. However, the parties in $G \setminus \{p\}$ cannot distinguish between $\mathcal{E}_2$ and $\mathcal{E}_3$ and hence, they give the same output in $\mathcal{E}_2$ and $\mathcal{E}_3$, a contradiction. $\square$

7.2.2 UP Broadcast with a Late-Joining Sender and Enough Time

Now that we have made clear that no protocol can handle the sender joining just one round before the parties of $\mathcal{P}_\mathcal{E}^1$ are about to terminate uncorrupted, one may wonder whether imposing an additional restriction to the joining time of the sender (apart from no party having terminated uncorrupted) may allow for a feasible solution to the problem.

For instance, if we assume that there exists a fixed upper bound r on the number of rounds that the sender may be late, then the solution to the problem becomes trivial: all parties remain silent for r rounds and then, start running a UP Broadcast protocol (like the one we presented in Section 5). Apparently, this protocol would require overall $r + |\mathcal{P}_\mathcal{E}^{R_T}|$ rounds so then all parties in $\mathcal{P}_\mathcal{E}^1$ that remain honest can safely terminate.

Thus, one may wonder whether there exists a protocol that allows the parties to terminate in less than $|\mathcal{P}_\mathcal{E}^{R_T}|$ rounds after the round when the sender joins the execution. Since we want to illuminate the profound reasons why this is impossible, we will prove that $\lfloor 2|\mathcal{P}_\mathcal{E}^{R_T}|/|\mathcal{H}_\mathcal{E}^{R_T}| \rfloor$ is a lower bound for Broadcast's round complexity even in a weaker model than the UP model that we have investigated so far. In fact, we will consider the setting where before the input value is given to the sender and the agreement process begins, the active parties may have already exchanged messages notifying each other about their will to participate. We will call this setting the *interactive-preprocessing* model and we will show that the desired lower bound for late-joining sender UP Broadcast protocols is implied by the corresponding lower bound in the *interactive-preprocessing* model.

7.2.3 UP Broadcast with Interactive Preprocessing

The *interactive-preprocessing* (IP) setting constitutes a relaxation of the UP setting that we described in Section 2.1. The main difference is that in the IP model, the sender is not given its input value at the beginning of the execution. Hence, the environment $\mathcal{Z}$ can activate parties at its discretion, which may communicate with each other for an arbitrary number of rounds before the actual Broadcast process begins. At some point, the sender s is given its input value and the parties are notified. Therefore, this setup phase can effectively help so-far honest parties to introduce themselves to each other, for as long as no information about the input value is provided to the sender.

Since we introduce this model exclusively for proving the desired lower bound, we assume that $\mathcal{Z}$ and $\mathcal{A}$ both terminate (i.e., stop activating/corrupting parties, respectively) in round R_T and that the input is given to the sender in round R_T as well. Thus, the definition of broadcast in the IP model naturally generalises the one of UP Broadcast:

Definition 7.7 (IP Broadcast). *Let $\kappa \in \mathbb{N}$ be a security parameter, let $\mathcal{U}$ be a (possibly exponential) set of parties, and let $s \in \mathcal{U}$. A protocol π is an **adaptively secure, IP Broadcast protocol with sender s** , if for every dynamic PPT environment $\mathcal{Z}$, every adaptive PPT adversary $\mathcal{A}$, and every input message $m_s \in \{0, 1\}^\ell$ with $\ell = O(\kappa)$, with overwhelming probability, an execution $\mathcal{E}$ of π with sender input m_s , where $\mathcal{Z}$ and $\mathcal{A}$ terminate by round R_T , has the following properties:*

- **Validity:** *Every party $p \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{R_T}$ terminates with output $m_p \in \{0, 1\}^\ell \cup \{\perp\}$, where $m_p = m_s$ if $s \in \mathcal{H}_\mathcal{E}^{R_T}$ and $m_p = \perp$ if $s \notin \mathcal{P}_\mathcal{E}^{R_T}$.*
- **Consistency:** *For all parties $p, q \in \mathcal{P}_\mathcal{E}^1 \cap \mathcal{H}_\mathcal{E}^{R_T}$ which output m_p and m_q , respectively, $m_p = m_q$.*
- **Termination:** *All parties in $\mathcal{P}_\mathcal{E}^1$ either terminate in polynomially (in κ) many rounds, or they get corrupted before they terminate.*

Once we have introduced the IP model, we will now show that any protocol that satisfies the properties of IP Broadcast requires at least $\left\lceil 2|\mathcal{P}_\varepsilon^{R_T}|/|\mathcal{H}_\varepsilon^{R_T}| \right\rceil$ rounds after the input is given to the sender. Our argument generalises the corresponding lower bound in the UP model from Section 6.

Lemma 7.8. *There exist an adaptive environment $\mathcal{Z}$ and an adaptive adversary $\mathcal{A}$ such that the following holds for every randomized IP Broadcast protocol π : In a random execution $\mathcal{E}$ with $\mathcal{Z}$ and $\mathcal{A}$, in which $\mathcal{Z}$ activates party-set $\mathcal{P}_\varepsilon^{R_T}$ by round R_T and $\mathcal{A}$ corrupts all but two parties, it holds that at least one party in $\mathcal{H}_\varepsilon^{R_T}$ runs for at least $|\mathcal{P}_\varepsilon^{R_T}|$ rounds after R_T , except with negligible probability (where the probability is over the coins of the environment, the adversary, and the honest parties).*

Proof. Let π be an IP Broadcast protocol and, without loss of generality, assume that the sender is p_1 . Given the security parameter κ , consider the set of parties $\mathcal{P} = \{p_1, \dots, p_\kappa\}$, and define the following two static environments:

- $\mathcal{Z}_1$ activates the parties in $\mathcal{P} \setminus \{p_1\}$ at the beginning of the execution and activates p_1 right before terminating in round R_T .
- $\mathcal{Z}_2$ activates the parties in $\mathcal{P} \setminus \{p_1\}$ at the beginning of the execution and terminates in round R_T .

Hence we can consider the same list of κ adversaries $\mathcal{A}_1, \dots, \mathcal{A}_\kappa$ as in the proof of Lemma 6.1 and also consider the same κ scenarios $S_1 \dots, S_\kappa$. Thus, we end up with the same configurations as the proof of Lemma 6.1.

So, let $b \leftarrow \{0, 1\}$ be a uniformly distributed bit. If we assume that in every execution $\mathcal{E}$ of π in scenarios $S_1, \dots, S_\kappa$, where the sender p_1 is given the input bit b in round R_T , all honest parties terminate in $|\mathcal{P}_\varepsilon^{R_T}| - 1$ rounds, then using the same arguments we used for the proof of Lemma 6.1, we have that the Claims 6.2, 6.3 and 6.4 both hold (the proofs are identical by replacing $\mathcal{P}_\varepsilon$ with $\mathcal{P}_\varepsilon^{R_T}$), which is a contradiction. $\square$

Theorem 7.9. *There exist an adaptive environment $\mathcal{Z}$ and an adaptive adversary $\mathcal{A}$ such that the following holds for every randomized IP Broadcast protocol π : In a random execution $\mathcal{E}$ with $\mathcal{Z}$ and $\mathcal{A}$, in which $\mathcal{Z}$ activates party-set $\mathcal{P}_\varepsilon^{R_T}$ by round R_T and $\mathcal{A}$ corrupts all but a subset of parties $\mathcal{H}_\varepsilon^{R_T}$, the honest parties in $\mathcal{H}_\varepsilon^{R_T}$ must run for at least $\left\lceil 2|\mathcal{P}_\varepsilon^{R_T}|/|\mathcal{H}_\varepsilon^{R_T}| \right\rceil$ rounds after R_T , except for negligible probability.*

Proof. Identical to the proof of Theorem 6.5. $\square$

7.2.4 Late-Joining Sender UP Broadcast Lower Bound

Using a simple reduction from IP Broadcast to late-joining sender UP Broadcast, we can prove the desired lower bound for the latter:

Lemma 7.10. *There does not exist any randomized late-joining sender UP Broadcast protocol π (according to Definition 7.5), such that in every execution $\mathcal{E}$ of π , if $\mathcal{Z}$ activates s in round $r \leq R_T$ (where R_T is the round when $\mathcal{Z}$ terminates), before any of the parties in $\mathcal{P}_\varepsilon^1$ terminates uncorrupted, then all parties in $\mathcal{P}_\varepsilon^1$ either terminate before round $r + \left\lceil 2|\mathcal{P}_\varepsilon^{R_T}|/|\mathcal{H}_\varepsilon^{R_T}| \right\rceil$ or they get corrupted before they terminate.*

Proof. Assume for the contrary that there exists such a late-joining sender UP Broadcast protocol π_{ls} , then, we can construct an IP Broadcast protocol $\pi_{\text{ip}}(\pi_{\text{ls}})$ by running a π_{ls} protocol where s participates as a late-joining sender.

In the IP model, we assume that $\mathcal{Z}$ and $\mathcal{A}$ terminate in round R_T when the input is given to the sender, and thus, by assumption, we have that all parties of $\mathcal{P}_\varepsilon^1$ that terminate uncorrupted,

terminate before round $R_T + \left\lceil 2|\mathcal{P}^{R_T}|/|\mathcal{H}^{R_T}| \right\rceil$. Moreover, in every execution of π_{ip} , if $s \in \mathcal{H}_{\mathcal{E}}^{R_T}$, then by validity of π_{is} , we have that all parties in $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{R_T}$ output m_s and that all parties of $\mathcal{P}_{\mathcal{E}}^1 \cap \mathcal{H}_{\mathcal{E}}^{R_T}$ output the same value, due to the consistency property of π_{is} . Therefore, π_{ip} is a randomized IP Broadcast protocol that terminates before round $R_T + \left\lceil 2|\mathcal{P}_{\mathcal{E}}^{R_T}|/|\mathcal{H}_{\mathcal{E}}^{R_T}| \right\rceil$, which contradicts Theorem 7.9. $\square$

References

- [ABdSFG08] Eduardo Adílio Pelinson Alchieri, Alysson Neves Bessani, Joni da Silva Fraga, and Fabíola Greve. Byzantine consensus with unknown participants. In Theodore P. Baker, Alain Bui, and Sébastien Tixeul, editors, *Proceedings of the 12th International Conference Principles of Distributed Systems OPODIS 2008*, volume 5401 of *Lecture Notes in Computer Science*, pages 22–40. Springer, 2008.
- [ACD⁺19] Ittai Abraham, T.-H. Hubert Chan, Danny Dolev, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Communication complexity of byzantine agreement, revisited. In Peter Robinson and Faith Ellen, editors, *38th ACM PODC*, pages 317–326. ACM, July / August 2019.
- [AD15a] Ittai Abraham and Danny Dolev. Byzantine agreement with optimal early stopping, optimal resilience and polynomial complexity. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC 2015, Portland, OR, USA, June 14-17, 2015*, pages 605–614. ACM, 2015.
- [AD15b] Marcin Andrychowicz and Stefan Dziembowski. PoW-based distributed cryptography with no trusted setup. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 379–399. Springer, Heidelberg, August 2015.
- [ADD⁺19] Ittai Abraham, Srinivas Devadas, Danny Dolev, Kartik Nayak, and Ling Ren. Synchronous byzantine agreement with expected $O(1)$ rounds, expected $O(n^2)$ communication, and optimal resilience. In Ian Goldberg and Tyler Moore, editors, *FC 2019*, volume 11598 of *LNCS*, pages 320–334. Springer, Heidelberg, February 2019.
- [BDDS92] Amotz Bar-Noy, Danny Dolev, Cynthia Dwork, and H. Raymond Strong. Shifting gears: Changing algorithms on the fly to expedite byzantine agreement. *Inf. Comput.*, 97(2):205–233, 1992.
- [Ben83] Michael Ben-Or. Another advantage of free choice: Completely asynchronous agreement protocols (extended abstract). In Robert L. Probert, Nancy A. Lynch, and Nicola Santoro, editors, *2nd ACM PODC*, pages 27–30. ACM, August 1983.
- [BGIK16] Amos Beimel, Ariel Gabizon, Yuval Ishai, and Eyal Kushilevitz. Distribution design. In Madhu Sudan, editor, *ITCS 2016*, pages 81–92. ACM, January 2016.
- [BGK⁺18] Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 913–930. ACM Press, October 2018.
- [BGP92] Piotr Berman, Juan A. Garay, and Kenneth J. Perry. Optimal early stopping in distributed consensus (extended abstract). In *Proceedings of the 6th International Workshop on Distributed Algorithms, WDAG ’92*, page 221–237, Berlin, Heidelberg, 1992. Springer-Verlag.
- [BIK17] Amos Beimel, Yuval Ishai, and Eyal Kushilevitz. Ad hoc PSM protocols: Secure computation without coordination. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part III*, volume 10212 of *LNCS*, pages 580–608. Springer, Heidelberg, April / May 2017.

- [BMTZ17] Christian Badertscher, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. Bitcoin as a transaction ledger: A composable treatment. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 324–356. Springer, Heidelberg, August 2017.
- [Bor96] Malte Borderding. Levels of authentication in distributed agreement. In *10th International Workshop on Distributed Algorithms WDAG*, pages 40–55, 1996.
- [Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- [Can04] Ran Canetti. Universally composable signature, certification, and authentication. In *17th IEEE Computer Security Foundations Workshop, (CSFW)*, pages 219–233. IEEE Computer Society, 2004.
- [CCL15] Ran Canetti, Asaf Cohen, and Yehuda Lindell. A simpler variant of universally composable security for standard multiparty computation. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 3–22. Springer, Heidelberg, August 2015.
- [CDL⁺24] Ran Cohen, Jack Doerner, Eysa Lee, Anna Lysyanskaya, and Lawrence Roy. An unstoppable ideal functionality for signatures and a modular analysis of the dolev-strong broadcast. *IACR Cryptol. ePrint Arch.*, page 1807, 2024.
- [CDM25] Ran Cohen, Poulami Das, and Tal Moran. Setup-free committee sampling with subquadratic communication. *Asiacrypt 2025 (to appear)*, 2025.
- [CFGN96] Ran Canetti, Uriel Feige, Oded Goldreich, and Moni Naor. Adaptively secure multi-party computation. In *28th ACM STOC*, pages 639–648. ACM Press, May 1996.
- [CGZ23] Ran Cohen, Juan A. Garay, and Vassilis Zikas. Completeness theorems for adaptively secure broadcast. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 3–38. Springer, Heidelberg, August 2023.
- [CLOS02] Ran Canetti, Yehuda Lindell, Rafail Ostrovsky, and Amit Sahai. Universally composable two-party and multi-party secure computation. In *34th ACM STOC*, pages 494–503. ACM Press, May 2002.
- [CPS20] T.-H. Hubert Chan, Rafael Pass, and Elaine Shi. Sublinear-round byzantine agreement under corrupt majority. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 246–265. Springer, Heidelberg, May 2020.
- [DKL25] Giovanni Deligios, Ivana Klasovita, and Chen-Da Liu-Zhang. Asymptotically optimal early termination for dishonest majority broadcast. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part V*, volume 15605 of *Lecture Notes in Computer Science*, pages 116–144. Springer, 2025.
- [DPS19] Phil Daian, Rafael Pass, and Elaine Shi. Snow white: Robustly reconfigurable consensus and applications to provably secure proof of stake. In Ian Goldberg and Tyler Moore, editors, *FC 2019*, volume 11598 of *LNCS*, pages 23–41. Springer, Heidelberg, February 2019.
- [DRS90] Danny Dolev, Rüdiger Reischuk, and H. Raymond Strong. Early stopping in byzantine agreement. *J. ACM*, 37(4):720–741, 1990.
- [DS83] Danny Dolev and H. Raymond Strong. Authenticated algorithms for Byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.

- [Fit02] Matthias Fitzi. *Generalized communication and security models in Byzantine agreement*. PhD thesis, ETH Zurich, 2002.
- [FLM86] Michael J. Fischer, Nancy A. Lynch, and Michael Merritt. Easy impossibility proofs for distributed consensus problems. *Distributed Comput.*, 1(1):26–39, 1986.
- [FM97] Peasech Feldman and Silvio Micali. An optimal probabilistic protocol for synchronous byzantine agreement. *SIAM Journal on Computing*, 26(4):873–933, 1997.
- [FN09] Matthias Fitzi and Jesper Buus Nielsen. On the number of synchronous rounds sufficient for authenticated byzantine agreement. In Idit Keidar, editor, *Distributed Computing, 23rd International Symposium, DISC 2009, Elche, Spain, September 23-25, 2009. Proceedings*, volume 5805 of *Lecture Notes in Computer Science*, pages 449–463. Springer, 2009.
- [GK20] Juan A. Garay and Aggelos Kiayias. SoK: A consensus taxonomy in the blockchain era. In Stanislaw Jarecki, editor, *CT-RSA 2020*, volume 12006 of *LNCS*, pages 284–318. Springer, Heidelberg, February 2020.
- [GKKO07] Juan A. Garay, Jonathan Katz, Chiu-Yuen Koo, and Rafail Ostrovsky. Round complexity of authenticated broadcast with a dishonest majority. In *48th FOCS*, pages 658–668. IEEE Computer Society Press, October 2007.
- [GKL17] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol with chains of variable difficulty. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 291–323. Springer, Heidelberg, August 2017.
- [GKL24] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol: Analysis and applications. *Journal of the ACM*, 71(4):1–49, 2024.
- [GKO⁺20] Juan A. Garay, Aggelos Kiayias, Rafail M. Ostrovsky, Giorgos Panagiotakos, and Vassilis Zikas. Resource-restricted cryptography: Revisiting MPC bounds in the proof-of-work era. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 129–158. Springer, Heidelberg, May 2020.
- [GKZ10] Juan A. Garay, Aggelos Kiayias, and Hong-Sheng Zhou. A framework for the sound specification of cryptographic tasks. In Andrew Myers and Michael Backes, editors, *CSF 2010 Computer Security Foundations Symposium*, pages 277–289. IEEE Computer Society Press, 2010.
- [GL05] Shafi Goldwasser and Yehuda Lindell. Secure multi-party computation without agreement. *Journal of Cryptology*, 18(3):247–287, July 2005.
- [HZ10] Martin Hirt and Vassilis Zikas. Adaptively secure broadcast. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 466–485. Springer, Heidelberg, May / June 2010.
- [IKP10] Yuval Ishai, Eyal Kushilevitz, and Anat Paskin. Secure multiparty computation with minimal interaction. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 577–594. Springer, Heidelberg, August 2010.
- [KK06] Jonathan Katz and Chiu-Yuen Koo. On expected constant-round protocols for byzantine agreement. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 445–462. Springer, Heidelberg, August 2006.
- [KW21] Pankaj Khanchandani and Roger Wattenhofer. Byzantine agreement with unknown participants and failures. In *35th IEEE International Parallel and Distributed Processing Symposium, IPDPS 2021*, pages 952–961. IEEE, 2021.
- [Lam83] Leslie Lamport. The weak byzantine generals problem. *Journal of the ACM*, 30(3):668–676, 1983.

- [LN24] Julian Loss and Jesper Buus Nielsen. Early stopping for any number of corruptions. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part III*, volume 14653 of *Lecture Notes in Computer Science*, pages 457–488. Springer, 2024.
- [LR23a] Andrew Lewis-Pye and Tim Roughgarden. Byzantine generals in the permissionless setting. In Foteini Baldimtsi and Christian Cachin, editors, *The 27th International Conference on Financial Cryptography and Data Security FC 2023, Part I*, volume 13950 of *Lecture Notes in Computer Science*, pages 21–37. Springer, 2023.
- [LR23b] Andrew Lewis-Pye and Tim Roughgarden. Permissionless consensus. *CoRR*, abs/2304.14701, 2023.
- [LSP82] Leslie Lamport, Robert E. Shostak, and Marshall C. Pease. The Byzantine generals problem. *ACM Trans. Program. Lang. Syst.*, 4(3):382–401, 1982.
- [MMR23] Dahlia Malkhi, Atsuki Momose, and Ling Ren. Towards practical sleepy bft. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 490–503, 2023.
- [MR22] Atsuki Momose and Ling Ren. Constant latency in sleepy consensus. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2295–2308. ACM Press, November 2022.
- [Per85] Kenneth J. Perry. Early stopping protocols for fault-tolerant distributed agreement. Technical report, USA, 1985.
- [PS17] Rafael Pass and Elaine Shi. The sleepy model of consensus. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part II*, volume 10625 of *LNCS*, pages 380–409. Springer, Heidelberg, December 2017.
- [PSL80] Marshall C. Pease, Robert E. Shostak, and Leslie Lamport. Reaching agreement in the presence of faults. *Journal of the ACM*, 27(2):228–234, 1980.
- [PSs17] Rafael Pass, Lior Seeman, and abhi shelat. Analysis of the blockchain protocol in asynchronous networks. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part II*, volume 10211 of *LNCS*, pages 643–673. Springer, Heidelberg, April / May 2017.
- [Rab83] Michael O. Rabin. Randomized byzantine generals. In *24th FOCS*, pages 403–409. IEEE Computer Society Press, November 1983.
- [Sch90] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Comput. Surv.*, 22(4):299–319, 1990.
- [WXDS20] Jun Wan, Hanshen Xiao, Srinivas Devadas, and Elaine Shi. Round-efficient byzantine broadcast under strongly adaptive and majority corruptions. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 412–456. Springer, Heidelberg, November 2020.
- [WXSD20] Jun Wan, Hanshen Xiao, Elaine Shi, and Srinivas Devadas. Expected constant round byzantine broadcast under dishonest majority. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 381–411. Springer, Heidelberg, November 2020.